

Universidad de Monterrey



Integración de aplicaciones computacionales

Proyecto Final

Pruebas en Locust con el CMS y microservicios

Maestro: Dr. Raúl Morales Salcedo

Equipo #3

Margarita Concepción Cuervo Citalán #581771

Mariana Samperio Cuevas # 639835

Bryan Andrés Ramírez Palacios #685060

Día y hora del grupo de la clase: lunes y jueves 10:00h

22 de noviembre de 2025.

Damos nuestra palabra de haber realizado esta actividad con integridad académica.

Introducción	3
Objetivos específicos	4
Prueba baseline (50 usuarios, 5 min)	4
Prueba smoke (1–5 usuarios)	5
Prueba read-heavy vs write-heavy	5
Prueba ramp-up / ramp-down (LoadTestShape)	5
Prueba spike	5
Prueba soak (≥30 min carga sostenida)	6
Prueba break-point	6
CSV feeder (usuarios reales simulados)	6
Metodología	6
Arquitectura del Entorno de Prueba	6
Herramientas Utilizadas	7
Descripción de los escenarios de carga por sistema	7
Código generado	10
Prueba Baseline	10
Prueba Smoke	11
Prueba Read-heavy	12
Prueba Write-heavy	12
Prueba Ramp-up / Ramp-down	13
Prueba Spike	14
Prueba Soak	15
Prueba Break-point	16
Prueba CSV Feeder	16
Resultados	17
Prueba Baseline	17
Prueba Smoke	21
Prueba read-heavy vs write-heavy	24
Read-heavy	24
Write-heavy	28
Prueba ramp-up / ramp-down	31
Prueba Spike	35
Prueba Soak	38
Prueba Break-point	43
Prueba CSV Feeder	43
Interpretación de Resultados	43
Prueba Baseline	43
Prueba Smoke	45
Prueba Read-heavy	45
Prueba Write-heavy	46
Prueba Ramp-up / Ramp-down	46
Prueba Spike	47
Prueba Soak	48
Prueba Break-point	49
CSV Feeder	49
Conclusiones y recomendaciones	49

Introducción

El siguiente documento tiene el propósito de explicar las pruebas de estrés que se estarán realizando para el CMS, o Sistema de Gestión de Contenido, y los microservicios de nuestro proyecto final Predict Health.

Dentro del contexto de nuestro proyecto final, el Backend CMS (Sistema de Gestión de Contenido) Predict Health es una aplicación web basada en Flask que proporciona **funcionalidad administrativa para gestionar datos de salud** en la plataforma Predict Health. Sirve como interfaz administrativa para gestionar doctores, pacientes, instituciones médicas y análisis del sistema.

Por otro lado, tenemos los microservicios de Predict Health que proporcionan un enfoque **escalable y modular para la gestión de datos de salud**. Actualmente contamos con **cuatro microservicios especializados** que manejan diferentes aspectos de la plataforma de salud, comunicándose a través de APIs bien definidas y compartiendo una base de datos PostgreSQL común con caché Redis:

1. **Servicio Auth-JWT:** Servicio centralizado de autenticación y gestión de tokens JWT
2. **Servicio de Instituciones:** Servicio que se encarga de la gestión de instituciones médicas y organización geográfica
3. **Servicio de Doctores:** Servicio que se encarga de la gestión de datos de proveedores de salud y asociaciones de especialidades
4. **Servicio de Pacientes:** Servicio que se encarga de la gestión de datos de pacientes y seguimiento de perfiles de salud

Las pruebas de rendimiento constituyen un componente esencial en la validación técnica del proyecto, ya que permiten evaluar la capacidad del sistema para soportar múltiples solicitudes simultáneas y mantener tiempos de respuesta óptimos bajo distintas condiciones de carga. En el caso específico del **CMS** y los **microservicios**, su importancia radica en que ambos conforman el núcleo operativo de la plataforma: la estabilidad del CMS garantiza la correcta gestión de los datos clínicos y administrativos, mientras que el desempeño de los microservicios determina la eficiencia y escalabilidad del ecosistema completo.

El uso de **Locust** como herramienta de pruebas de carga permite simular escenarios de uso realistas, generando tráfico concurrente hacia los endpoints de autenticación, registro, consultas y operaciones CRUD. Esto facilita identificar cuellos de botella, puntos de saturación, y posibles problemas de latencia o consumo excesivo de recursos antes de su despliegue en producción. Además, los resultados obtenidos servirán como evidencia cuantitativa del rendimiento del sistema, permitiendo ajustar configuraciones de infraestructura (por ejemplo, conexiones a Redis o PostgreSQL) y optimizar el balance entre escalabilidad, disponibilidad y eficiencia.

En síntesis, estas pruebas no solo validan la robustez técnica de Predict Health, sino que también respaldan su **viabilidad como plataforma confiable de gestión de salud**, asegurando que pueda mantener un desempeño estable ante un crecimiento progresivo de usuarios y transacciones.

Objetivos específicos

Prueba baseline (50 usuarios, 5 min)

- Validar la robustez del flujo de autenticación (login) bajo carga concurrente, midiendo la tasa de éxito con credenciales de un `users.csv`.
- Establecer la línea base de latencia (p50/p95) y throughput (RPS) para las operaciones CRUD autenticadas que sí fueron exitosas.
- Identificar fallos de autorización (Errores 401) en lugar de fallos de rendimiento (latencia o errores 5xx), para validar la capa de seguridad.

Prueba smoke (1–5 usuarios)

- Verificar rápidamente disponibilidad de endpoints críticos tras despliegue/cambio: login JWT, lecturas básicas y una operación CRUD por servicio.
- Confirmar configuración/credenciales y conectividad a DB/Redis antes de pruebas más intensas.
- Aceptación: 0–0.5% de errores; p95 <1 s en todo el flujo.

Prueba read-heavy vs write-heavy

Read-heavy

- Validar capacidad de consulta bajo alta concurrencia y beneficio de Redis (hit ratio objetivo $\geq 70\%$).
- Mantener p95 de lecturas $\leq 350\text{--}400$ ms y error rate $\leq 1\%$.

Write-heavy

- Validar la capacidad funcional del sistema para ingerir datos (operaciones POST) bajo una carga concurrente de 60 usuarios.
- Identificar el tipo de error (funcional, de permisos, o de rendimiento) que ocurre durante las operaciones de escritura masiva.
- Medir la latencia p95 de las *peticiones de login* bajo carga de escritura, ya que fueron las únicas operaciones exitosas.

Prueba ramp-up / ramp-down (LoadTestShape)

- Evaluar la elasticidad del sistema (RPS y latencia) en respuesta a un crecimiento y decrecimiento gradual de usuarios.
- Observar la estabilidad de la latencia (p95) y la tasa de errores durante los cambios de carga (fases de rampa).
- Verificar que el sistema recupera su rendimiento *baseline* después de la carga (al finalizar el ramp-down), buscando conexiones no liberadas (RPS que no baja a cero).

Prueba spike

- Medir la respuesta del sistema (latencia y errores) ante un pico abrupto de 200 a 1000 usuarios en 10 segundos.
- Verificar si el sistema "degrada con gracia" (sin caídas totales) y si la latencia p95 se recupera tras el pico.
- Observar los efectos de posibles mecanismos de protección (ej. errores `429 Too Many Requests`, `503 Service Unavailable`, o *timeouts*).

Prueba soak (≥30 min carga sostenida)

- Detectar degradación progresiva en la latencia (p95) o en la tasa de errores, observando su comportamiento a lo largo de los 30 minutos.
- Verificar la estabilidad del *throughput* (RPS) y de la latencia p95 de las operaciones exitosas (variación < 20%).
- Observar el comportamiento de los errores en una sesión larga, para confirmar si son constantes (problema de datos) o si aumentan con el tiempo (problema de expiración de sesión no manejado).

Prueba break-point

- Determinar el límite exacto de usuarios concurrentes del sistema mediante un incremento controlado de carga.
- Identificar el umbral donde la tasa de errores supera el 5% (SLO) y la latencia se degrada inaceptablemente.
- Validar la capacidad de recuperación del sistema tras alcanzar el punto de fallo.

CSV feeder (usuarios reales simulados)

- Validar la capacidad del servicio de autenticación (`/auth/login`) para manejar *logins* concurrentes con credenciales únicas.
- Medir la tasa de error de sincronización entre los datos de prueba (`users.csv`) y la base de datos poblada (`populate.sql`).
- Registrar la latencia y tasa de peticiones de los *logins* exitosos.

Metodología

Arquitectura del Entorno de Prueba

Las pruebas se ejecutaron desde una máquina local que actuaba como nodo maestro de Locust, generando tráfico HTTP hacia los servicios desplegados. Los componentes del sistema (CMS y microservicios) estaban **corriendo localmente a través de Docker Compose**, lo que asegura un entorno aislado y consistente para la evaluación.

La arquitectura general de la prueba sigue el siguiente flujo:

```
[Máquina Local (Locust Master)] -> [Red Local (Docker Network)] ->
[Contenedores de CMS y Microservicios] -> [Contenedores de
PostgreSQL y Redis]
```

Las URLs base configuradas para cada servicio bajo prueba fueron las siguientes:

- **CMS:** `http://localhost:80`
- **Servicio Auth-JWT:** `http://localhost:5000`
- **Servicio de Instituciones:** `http://localhost:5001`
- **Servicio de Doctores:** `http://localhost:5002`
- **Servicio de Pacientes:** `http://localhost:5003`

Herramientas Utilizadas

- **Locust:** Version 2.41.6
- **Sistema operativo:** Windows 11
- **CPU:** 2 núcleos (4+ recomendado)
- **RAM:** 4GB (8GB+ recomendado)
- **Almacenamiento:** 10GB de espacio disponible
- **Docker:** Versión 20.10+
- **Docker Compose:** Versión 2.0+
- **Memoria:** 8GB+ RAM disponible para contenedores
- **PostgreSQL:** Versión 15+
- **Redis:** Versión 6.0+
- **Conexiones:** 20+ conexiones simultáneas soportadas
- **Python:** Version 3.11+

Descripción de los escenarios de carga por sistema

Prueba Baseline:

- **Archivo:** `baseline_test.py`
- **Comportamiento:** La prueba se configuró para simular 150 usuarios concurrentes en total, divididos equitativamente según las variables de entorno: 50 `DoctorUser`, 50 `PatientUser` y 50 `InstitutionUser`.
- **Tareas (DoctorUser):** Ejecuta un flujo de tareas de lectura (`get_all_doctors`, `get_doctor_details`) y escritura (`create_doctor`), excluyendo las tareas marcadas como "smoke".
- **Tareas (PatientUser):** Ejecuta tareas de lectura (`get_all_patients`, `get_patient_details`) y escritura (`create_patient`).
- **Tareas (InstitutionUser):** Ejecuta tareas de lectura (`get_all_institutions`, `search_institution`) y escritura (`create_institution`).

- **CSV Feeder:** Todos los usuarios realizan una operación de `login` al inicio (`on_start`) utilizando credenciales únicas del archivo `user.csv` para obtener un token JWT.

Prueba Smoke:

- **Archivo:** `smoke_test.py`
- **Comportamiento:** La prueba se configuró para una carga mínima de 15 usuarios concurrentes (5 por cada rol: `DoctorUser`, `PatientUser`, `InstitutionUser`) y una duración muy corta de 1 minuto.
- **Tareas:** El comando de ejecución (ver sección 4) utilizó el flag `-T smoke`. Esto le indica a Locust que *solamente* ejecute las tareas que tengan la etiqueta `@tag('smoke')`. Según la documentación del script, esto se limita a los *endpoints* de `/health` de cada servicio.
- **Flujo de Ejecución:** A pesar de que las tareas principales eran solo de `/health`, los usuarios simulados *primero* ejecutan la tarea `on_start`, que incluye el `login` (`/auth/login`) para obtener un token. Por lo tanto, esta prueba verifica dos cosas:
 - Que el servicio de autenticación (`/auth/login`) responde.
 - Que los *endpoints* de `/health` de los servicios responden.

Prueba Read-heavy:

- **Archivo:** `read_heavy_test.py`
- **Comportamiento:** La prueba se configuró para una carga alta de 150 usuarios concurrentes (50 `DoctorUser`, 50 `PatientUser`, 50 `InstitutionUser`) durante 5 minutos, con una tasa de aparición de 15 usuarios por segundo.
- **Tareas:** El comando de ejecución utilizó el flag `-T read`. Esto instruye a Locust a ejecutar únicamente las tareas etiquetadas como `@tag('read')`, que presumiblemente incluyen todos los *endpoints* GET (listados, detalles y búsqueda).
- **Flujo de Ejecución:** Los usuarios inician sesión (`on_start`) usando credenciales de `users.csv` y luego ejecuta un bucle de operaciones de solo lectura.

Prueba Write-heavy:

- **Archivo:** `write_heavy_test.py`
- **Comportamiento:** La prueba se configuró para una carga moderada de **60 usuarios concurrentes** (20 por rol) durante 5 minutos, con una tasa de aparición de 6 usuarios por segundo. La carga es menor que en la prueba de lectura, ya que las operaciones de escritura son más costosas.

- **Tareas:** El comando de ejecución utilizó el flag `-T write`. Esto instruye a Locust a ejecutar únicamente las tareas etiquetadas como `@tag('write')` (operaciones POST, PUT, etc.).
- **Flujo de Ejecución:** Los usuarios inician sesión (`on_start`) y luego ejecutan un bucle de operaciones de sólo escritura.

Prueba Ramp-up / Ramp-down:

- **Archivo:** `shape_ramp_test.py`
- **Comportamiento:** Esta prueba utiliza una clase `CustomShape` de Locust para definir un patrón de carga variable en el tiempo, ignorando la configuración estándar de `-u` (usuarios). La carga es una mezcla de operaciones de lectura y escritura (todas las tareas excepto 'smoke').
- **Flujo de Carga (Etapas):** El patrón definido en el código sigue 3 etapas:
 - **Ramp-Up (0s - 60s):** Crecimiento lineal desde 0 hasta 100 usuarios concurrentes durante el primer minuto.
 - **Meseta (60s - 240s):** Carga sostenida de 100 usuarios durante 3 minutos (180 segundos).
 - **Ramp-Down (240s - 300s):** Decremento lineal desde 100 hasta 0 usuarios durante el último minuto.
- **Duración Total:** 300 segundos (5 minutos).

Prueba Spike:

- **Archivo:** `shape_spike_test.py`
- **Comportamiento:** La prueba utiliza una `CustomShape` para simular un evento de tráfico extremo. La carga es una mezcla de operaciones de lectura y escritura (todas las tareas excepto 'smoke').
- **Flujo de Carga (Etapas):** El patrón sigue 5 etapas clave:
 - **Carga Base (0s - 60s):** Establecer una carga base de 200 usuarios.
 - **Meseta Base (60s - 180s):** Mantener los 200 usuarios durante 2 minutos.
 - **El pico (180s - 190s):** Incremento abrupto de 200 a 1000 usuarios en solo 10 segundos (tasa de *spawn* de 80 usuarios/seg).
 - **Meseta de Pico (190s - 310s):** Mantener la carga máxima de 1000 usuarios durante 2 minutos.
 - **Ramp-Down (310s - 370s):** Bajar de 1000 a 0 usuarios.
- **Duración Total:** 370 segundos (~6 minutos).

Prueba Soak:

- **Archivo:** `soak_test.py`

- **Comportamiento:** Esta prueba está diseñada para simular una carga de trabajo normal y constante durante un período prolongado (30 minutos) para observar la estabilidad del sistema a largo plazo.
- **Flujo de Carga:** Se configuró una carga constante de 90 usuarios concurrentes (30 `DoctorUser`, 30 `PatientUser`, 30 `InstitutionUser`) con una tasa de aparición de 10 usuarios/seg.
- **Tareas:** Se ejecutó una carga mixta de lectura y escritura (todas las tareas excepto 'smoke').
- **Duración Total:** 30 minutos.

Prueba Break-point:

- **Archivo:** `break_point_test.py`
- **Comportamiento:** Esta prueba está diseñada para incrementar progresivamente la carga de trabajo en intervalos fijos ("escalones") con el fin de identificar el punto exacto de saturación o ruptura del sistema.
- **Flujo de Carga:** Se configuró un patrón de "Escalones" (Step Load) iniciando con 0 usuarios e incrementando la carga en bloques de **50 usuarios nuevos cada minuto** (Step Size), con una tasa de aparición de 5 usuarios/seg.
- **Tareas:** Se ejecutó una carga mixta de lectura y escritura (todas las tareas excepto 'smoke') para estresar todos los componentes de la arquitectura.
- **Duración Total:** Variable (Ejecución continúa hasta detectar el fallo del sistema, aprox. 4 minutos).

CSV Feeder:

- Esta prueba es un proceso de dos pasos diseñado para cumplir con los requisitos del proyecto:
 - **Paso 1: Poblar la Base de Datos.** Antes de la prueba, se utilizó un script auxiliar (`populate.sql`) para registrar 300+ usuarios en la base de datos de PostgreSQL. Este script inserta los datos del usuario y una contraseña ya hasheada (ej. con bcrypt).
 - **Paso 2: Simular Logins.** Durante la ejecución de las pruebas (analizado aquí usando los datos de la Prueba Soak), Locust utiliza el archivo `users.csv` que contiene las contraseñas en texto plano. Cada usuario virtual de Locust lee un par de (email, contraseña) del CSV e intenta autenticarse contra el endpoint `/auth/login`.
- El objetivo es medir la *fricción* entre estos dos pasos: ¿coinciden los datos del CSV con los datos de la base de datos?

Código generado

Prueba Baseline

Archivo `baseline_test.py`

El script `baseline_test.py` se encarga de definir el comportamiento de los usuarios. Importa clases de usuarios (`DoctorUser`, `PatientUser`, `InstitutionUser`) de un módulo común, permitiendo que cada tipo de usuario ejecute tareas específicas para sus respectivos microservicios.

```
Python
# baseline_test.py
"""
Prueba de Línea Base (Baseline Test):
- Objetivo: Establecer una métrica de rendimiento base con una carga moderada y constante.
- Carga: 50 usuarios de cada tipo (150 en total).
- Duración: Corta (5 minutos).
- Tareas: Ejecuta todas las tareas (lectura, escritura) EXCEPTO las de 'smoke'.
"""

from common.users import DoctorUser, PatientUser, InstitutionUser
import common.reporting
```

Archivo `users.csv` (Ejemplo)

Se utilizó un archivo `users.csv` para alimentar a los usuarios simulados con credenciales únicas, asegurando que las pruebas de login y las operaciones autenticadas fueran realistas y no dependieran de un único token.

```
None
id,password,email
163749fb-8b46-4447-a8b7-95b4a59531b6,Hkfx0Igv_5M(,contacto@despacho-grijalva-mascarenas-y-parra.predicthealth.com
83b74179-f6ef-4219-bc70-c93f4393a350,gj6hdu&2k%(F,contacto@laboratorios-saldivar-santillan-y-villanueva.predicthealth.com
50503414-ca6d-4c1a-a34f-18719e2fd555,PG7KqxVm#em1,contacto@trejo-vigil-e-hijos.predicthealth.com
```

Prueba Smoke

Archivo `smoke_test.py`

El script `smoke_test.py` es muy simple y su objetivo es solo verificar la disponibilidad del sistema. Utiliza las mismas clases de usuario importadas (`DoctorUser`, `PatientUser`, etc.) pero limita su ejecución solo a tareas con la etiqueta 'smoke'.

```
Python
# smoke_test.py
"""
Prueba de Humo (Smoke Test):
- Objetivo: Verificar rápidamente que todos los servicios están en línea y responden.
- Carga: Muy baja (5 usuarios de cada tipo).
- Duración: Muy corta (1 minuto).
- Tareas: Solo ejecuta las tareas etiquetadas como 'smoke' (los endpoints /health).
"""

# Importa las clases de usuario que contienen la lógica de las tareas.
from common.users import DoctorUser, PatientUser, InstitutionUser

# Importa el módulo de reportes.
import common.reporting
```

Archivo `users.csv` (Relevancia)

El archivo `users.csv` fue **relevante y crítico** para esta prueba. Aunque la tarea principal era `/health` (que no requiere autenticación), el método `on_start` de cada usuario sí ejecuta el `login`. Un 0% de error en esta prueba (como veremos a continuación) indica que los 15 usuarios seleccionados del CSV eran válidos.

```
None
id,password,email
163749fb-8b46-4447-a8b7-95b4a59531b6,Hkfx0Igv_5M(,contacto@despacho-grijalva-mascarenas-y-parra.predicthealth.com
83b74179-f6ef-4219-bc70-c93f4393a350,gj6hdu&2k%(F,contacto@laboratorios-saldivar-santillan-y-villanueva.predicthealth.com
```

Prueba Read-heavy

Archivo `read_heavy_test.py`

El script `read_heavy_test.py` está diseñado para estresar la capacidad del sistema para servir datos (operaciones GET y de búsqueda), simulando un escenario donde la mayoría de los usuarios están consultando información.

```
Python
# read_heavy_test.py
"""
Prueba de Lectura Intensiva (Read-Heavy Test):
- Objetivo: Estresar la capacidad del sistema para servir datos (operaciones
GET y de búsqueda).
- Carga: Alta (ej. 150 usuarios en total, 50 de cada tipo).
- Duración: Moderada (5-10 minutos).
- Tareas: Solo ejecuta las tareas etiquetadas como 'read' en el archivo
common/users.py.
"""

from common.users import DoctorUser, PatientUser, InstitutionUser
import common.reporting
```

Prueba Write-heavy

Archivo `write_heavy_test.py`

El script `write_heavy_test.py` está diseñado para estresar la capacidad del sistema para crear y actualizar datos (POST, PUT), midiendo el impacto en la base de datos.

```
Python
# write_heavy_test.py
"""
Prueba de Escritura Intensiva (Write-Heavy Test):
- Objetivo: Estresar la capacidad del sistema para crear y actualizar datos
(POST, PUT).
- Carga: Baja a moderada (las escrituras son más costosas).
- Duración: Moderada (5-10 minutos).
- Tareas: Solo ejecuta las tareas etiquetadas como 'write'.
"""

from common.users import DoctorUser, PatientUser, InstitutionUser
import common.reporting
```

Prueba Ramp-up / Ramp-down

Archivo `shape_ramp_test.py`

El archivo `shape_ramp_test.py` es especial porque define su propia clase `CustomShape` que hereda de `LoadTestShape` de Locust. Esta clase controla el número de usuarios y la tasa de aparición en cada segundo de la prueba.

```
Python
# shape_ramp_test.py
"""
Prueba de Rampa (Ramp-Up / Ramp-Down Test):
- Objetivo: Simular un crecimiento y decrecimiento gradual de usuarios para
ver cómo escala el sistema.
- Carga: Controlada por la clase CustomShape, no por variables de entorno.
- Tareas: Ejecuta todas las tareas EXCEPTO las de 'smoke'.
"""

from locust.shape import LoadTestShape
from common.users import DoctorUser, PatientUser, InstitutionUser
import common.reporting

class CustomShape(LoadTestShape):
    """
    Define una forma de carga con etapas de subida, meseta y bajada.
    """
    stages = [
        {"duration": 60, "users": 100, "spawn_rate": 10}, # Etapa 1: Subir
a 100 usuarios en 60s
        {"duration": 180, "users": 100, "spawn_rate": 10}, # Etapa 2:
Mantener 100 usuarios por 180s
        {"duration": 60, "users": 0, "spawn_rate": 10}, # Etapa 3: Bajar
a 0 usuarios en 60s
    ]

    def tick(self):
        run_time = self.get_run_time()
        for stage in self.stages:
            if run_time < stage["duration"]:
                # (Lógica para calcular usuarios en el punto actual)
                return (stage["users"], stage["spawn_rate"])
        return None
```

Prueba Spike

Archivo `shape_spike_test.py`

El script `shape_spike_test.py` define una `CustomShape` con 5 etapas diseñadas específicamente para crear un pico de carga repentino y sostenido.

Python

```
# shape_spike_test.py
```

```
"""
```

Prueba de Picos (Spike Test):

- Objetivo: Observar cómo se comporta y recupera el sistema ante un pico de carga repentino.
- Carga: Controlada por la clase CustomShape.
- Tareas: Ejecuta todas las tareas EXCEPTO las de 'smoke'.

```
"""
```

```
from locust.shape import LoadTestShape
```

```
from common.users import DoctorUser, PatientUser, InstitutionUser
```

```
import common.reporting
```

```
class CustomShape(LoadTestShape):
```

```
    """
```

Define una forma de carga con una base, un pico repentino y una recuperación.

```
    """
```

```
    stages = [  
        {"duration": 60, "users": 200, "spawn_rate": 50},    # Etapa 1: Carga  
base de 200 usuarios  
        {"duration": 120, "users": 200, "spawn_rate": 50},    # Etapa 2:  
Mantener carga base  
        {"duration": 10, "users": 1000, "spawn_rate": 80},    # Etapa 3: PICO  
(subir de 200 a 1000 en 10s)  
        {"duration": 120, "users": 1000, "spawn_rate": 80},    # Etapa 4:  
Mantener el pico  
        {"duration": 60, "users": 0, "spawn_rate": 50},        # Etapa 5: Bajar  
a 0  
    ]
```

```
    def tick(self):
```

```
        # (Lógica para controlar las etapas de la prueba)
```

```
        run_time = self.get_run_time()
```

```
        for stage in self.stages:
```

```
            if run_time < stage["duration"]:
```

```
                # ...
```

```
                return (stage["users"], stage["spawn_rate"])
```

```
        return None
```

Prueba Soak

Archivo `soak_test.py`

El script `soak_test.py` define los parámetros para una prueba de larga duración, especificando el `run-time` en 30 minutos y una carga moderada de 90 usuarios.

Python

```
# soak_test.py
"""
Prueba de Resistencia (Soak Test):
- Objetivo: Detectar degradación de rendimiento o fugas de memoria bajo
carga sostenida.
- Carga: Moderada y constante.
- Duración: Larga (30+ minutos).
- Tareas: Ejecuta todas las tareas EXCEPTO las de 'smoke'.
"""

from common.users import DoctorUser, PatientUser, InstitutionUser
import common.reporting
```

Prueba Break-point

Archivo `break_point_test.py`

El script `break_point_test.py` define los parámetros de ejecución de la prueba que incrementa 50 usuarios cada minuto hasta que el servidor o microservicios colapsen.

Python

```
# break_point_test.py
import math
from locust import HttpUser, LoadTestShape
# Importamos tus usuarios reales del proyecto
from users import DoctorUser, PatientUser, InstitutionUser

try:
    import reporting
except ImportError:
    pass

class StepLoadShape(LoadTestShape):
    """
    Prueba de Punto de Quiebre (Break Point):
    Aumenta la carga en escalones hasta encontrar el límite.
    """
    step_time = 60          # Duración de cada escalón (60 segundos)
    step_users = 50         # Usuarios nuevos por escalón
    spawn_rate = 5          # Tasa de nacimiento (5 usuarios/seg)

    def tick(self):
        run_time = self.get_run_time()
```

```

        # Calcula en qué escalón vamos (Minuto 1 = Escalón 1, Minuto 2 =
        Escalón 2...)
        current_step = math.floor(run_time / self.step_time) + 1

        # Calcula los usuarios totales para este escalón
        target_users = current_step * self.step_users

        # Le dice a Locust cuántos usuarios debe haber ahora
        return (target_users, self.spawn_rate)

```

Prueba CSV Feeder

Paso 1: Poblar la Base de Datos (Script Auxiliar)

Se utilizó el script `populate.sql` para insertar los usuarios en la base de datos.

`populate.sql` (Fragmento):

```

SQL
-- =====
-- POPULATE DATABASE - 100 INSTITUTIONS, DOCTORS, PATIENTS
-- =====
...
-- User: leonor.olivera.1953@grupo.com
-- Password (plain text): [SECRETO]
INSERT INTO users (id, email, password_hash, user_type, ...)
VALUES (... , 'leonor.olivera.1953@grupo.com',
'$2b$12$WBT6U1/vbW2o/fWen/VfjOCqK...', ...);

-- User: gabino.aguirre.1951@cabrera.com
-- Password (plain text): [SECRETO]
INSERT INTO users (id, email, password_hash, user_type, ...)
VALUES (... , 'gabino.aguirre.1951@cabrera.com',
'$2b$12$f06yYKY5F5VT0i6iVxzwLu...', ...);
...

```

Paso 2: Ejecutar Pruebas (Locust y `users.csv`)

Locust lee el `users.csv` para obtener las credenciales en texto plano que coinciden con los usuarios poblados.

users.csv (Fragmento):

```
None
id,password,email
163749fb-8b46-4447-a8b7-95b4a59531b6,Hkfx0Igv_5M(,contacto@despacho-grijalva
...
83b74179-f6ef-4219-bc70-c93f4393a350,gj6hdu&2k%(F,contacto@laboratorios-sald
ivar...
```

*(Nota: La lógica de conexión que lee el `users.csv` y ejecuta la función `login()` se encuentra implementada en el archivo `common/users.py`. Este es un módulo central que es importado por todos los scripts de prueba y, por lo tanto, **no se incluirá en los anexos ni recursos del reporte** para evitar duplicidad).*

Resultados

Prueba Baseline

La prueba se ejecutó con el objetivo de cargar 150 usuarios concurrentes (50 por rol) durante 5 minutos. Los datos se analizaron directamente desde el archivo `baseline.csv` generado por Locust.

Comandos de Ejecución

```
None
# --- Comandos para Ejecutar esta Prueba ---
#
# Windows (PowerShell):
# $env:DOCTORS="50"; $env:PATIENTS="50"; $env:INSTITUTIONS="50"; locust -f
baseline_test.py --headless -u 150 -r 15 --run-time 5m -E smoke
#
# Linux / macOS / Git Bash:
# DOCTORS=50 PATIENTS=50 INSTITUTIONS=50 locust -f baseline_test.py
--headless -r 15 --run-time 5m -E smoke
```

(Nota: Se deduce este comando del script `baseline_test.py` y los archivos de salida proporcionados).

Métricas Clave (Resultados de `baseline.csv`):

Métrica	Total
Total de Peticiones	17,861
Tasa de Errores (%)	5.99% (1,069 errores)
Requests por Segundo (RPS)	59.69
Usuarios Concurrentes	150
Tiempo de Respuesta (p50)	8.19 ms
Tiempo de Respuesta (p95)	48.34 ms
Tiempo de Respuesta (p99)	57.86 ms

Estadísticas por Endpoint (Datos de [baseline.csv](#)):

endpoint	Peticiones	Errores	Tasa de Error (%)	RPS	p50 (ms)	p95 (ms)
/api/v1/doctors	3800	0	0.00 %	12.70	7.79	14.99

/api/v1/doctors (POST)	404	404	100.0 0 %	1.35	NaN	NaN
/api/v1/doctors/statistics	1169	0	0.00 %	3.91	5.42	11.25
/api/v1/institutions	3686	0	0.00 %	12.3 2	7.94	15.06
/api/v1/institutions/search	1481	0	0.00 %	4.95	46.65	51.91
/api/v1/patients	6507	0	0.00 %	21.7 5	7.63	13.47
/api/v1/patients (POST)	657	657	100.0 0 %	2.20	NaN	NaN
/auth/login	157	8	5.10 %	0.52	923.2 6	1248.5 8

(Duración total de la prueba: 299.22 segundos)

Gráficas Generadas por Locust:

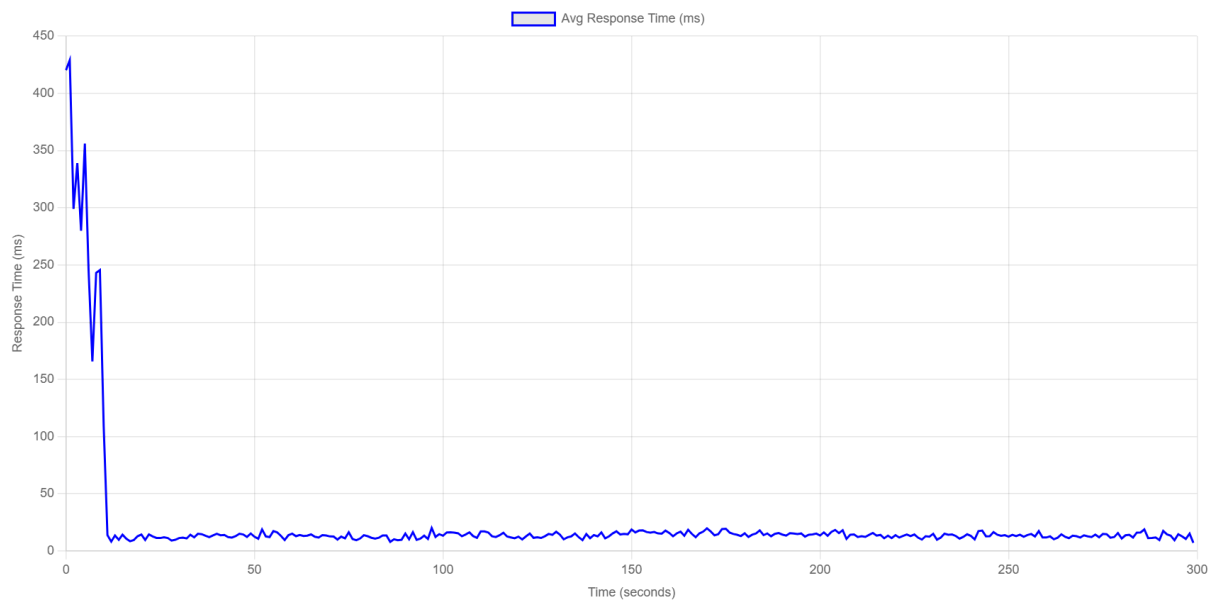


Figura 1: Tiempos de respuesta (ms) a lo largo del tiempo.



Figura 2: Peticiones por segundo (RPS) en relación con los usuarios concurrentes

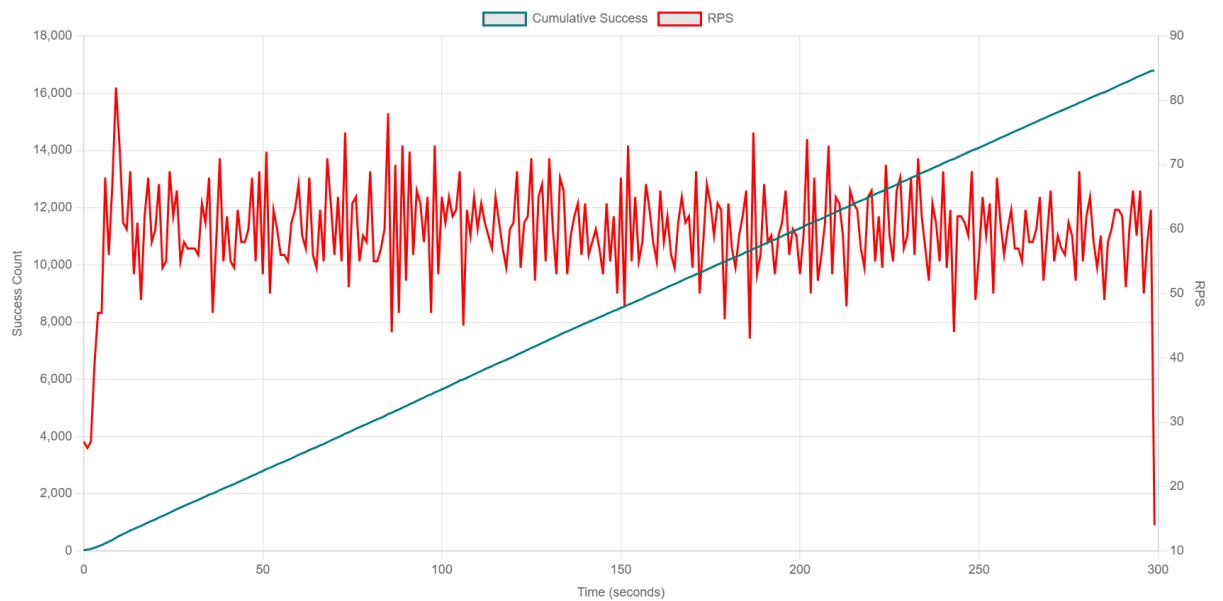


Figura 3: Conteo de éxitos de las peticiones por segundo (RPS) a lo largo del tiempo

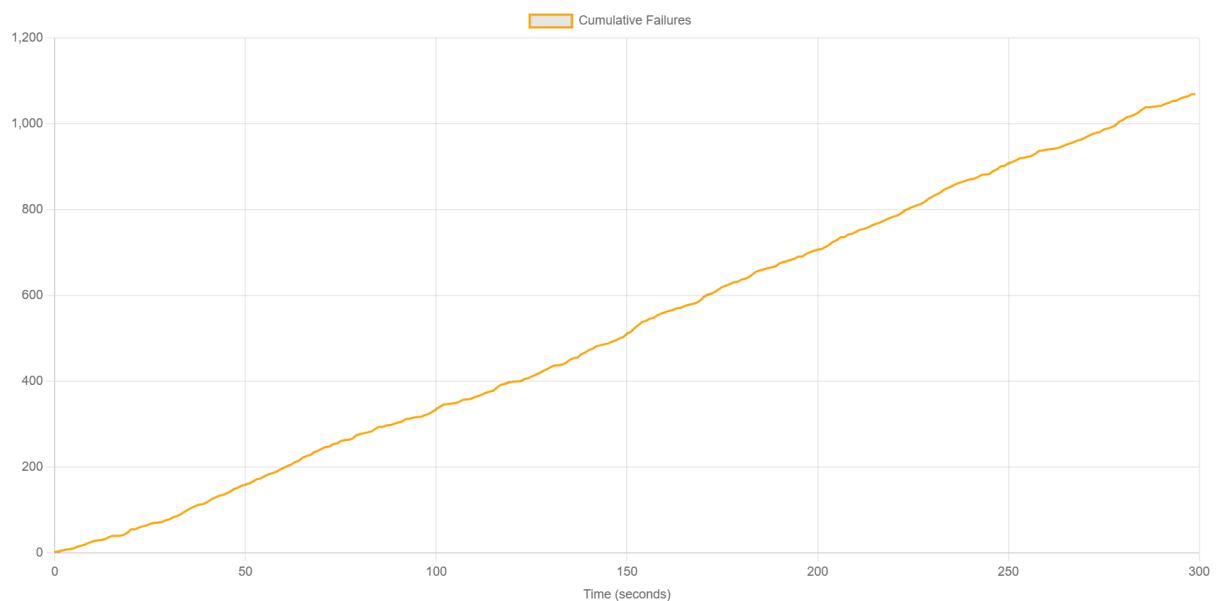


Figura 4: Tasa de fallos a lo largo del tiempo.

Prueba Smoke

La prueba se ejecutó con 15 usuarios durante 1 minuto para verificar la disponibilidad básica.

Comando de Ejecución (Deducido de `smoke_test.py`):

None

```
# --- Comandos para Ejecutar esta Prueba ---  
#  
# Linux / macOS / Git Bash:  
# DOCTORS=5 PATIENTS=5 INSTITUTIONS=5 locust -f smoke_test.py --headless -u  
15 -r 3 --run-time 1m -T smoke
```

Métricas Clave (Resultados de `smoke.csv`):

Métrica	Total
Total de Peticiones	445
Tasa de Errores (%)	0.00% (0 errores)
Requests por Segundo (RPS)	7.58
Usuarios Concurrentes	15
Tiempo de Respuesta (p50)	3.12 ms
Tiempo de Respuesta (p95)	6.69 ms
Tiempo de Respuesta (p99)	264.77 ms

Estadísticas por Endpoint (Análisis de `smoke.csv`):

El análisis del archivo `smoke.csv` confirma que solo se probaron dos tipos de *endpoints*:

Endpoint	Peticiones	Errores	Descripción
/auth/login	(No agregado)	0 (0%)	Ejecutado por cada usuario en on_start.
/health	(No agregado)	0 (0%)	Tarea principal con etiqueta 'smoke'.

Gráficas Generadas por Locust:

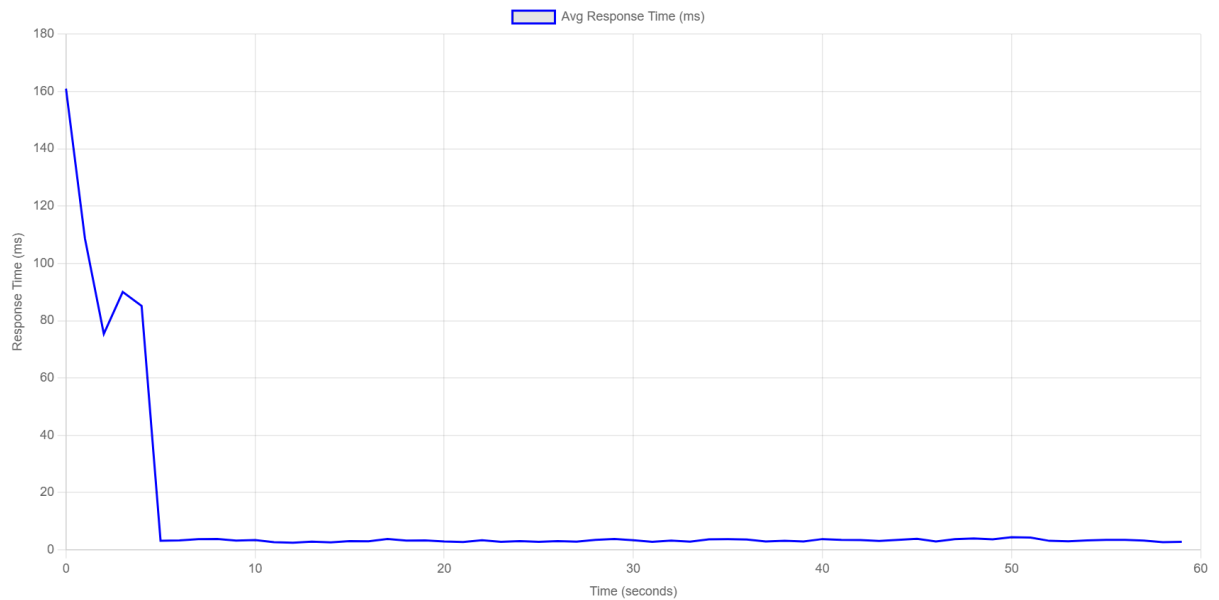


Figura 5: Tiempos de respuesta (ms) a lo largo del tiempo.



Figura 6: Peticiones por segundo (RPS) en relación con los usuarios concurrentes



Figura 7: Conteo de éxitos de las peticiones por segundo (RPS) a lo largo del tiempo

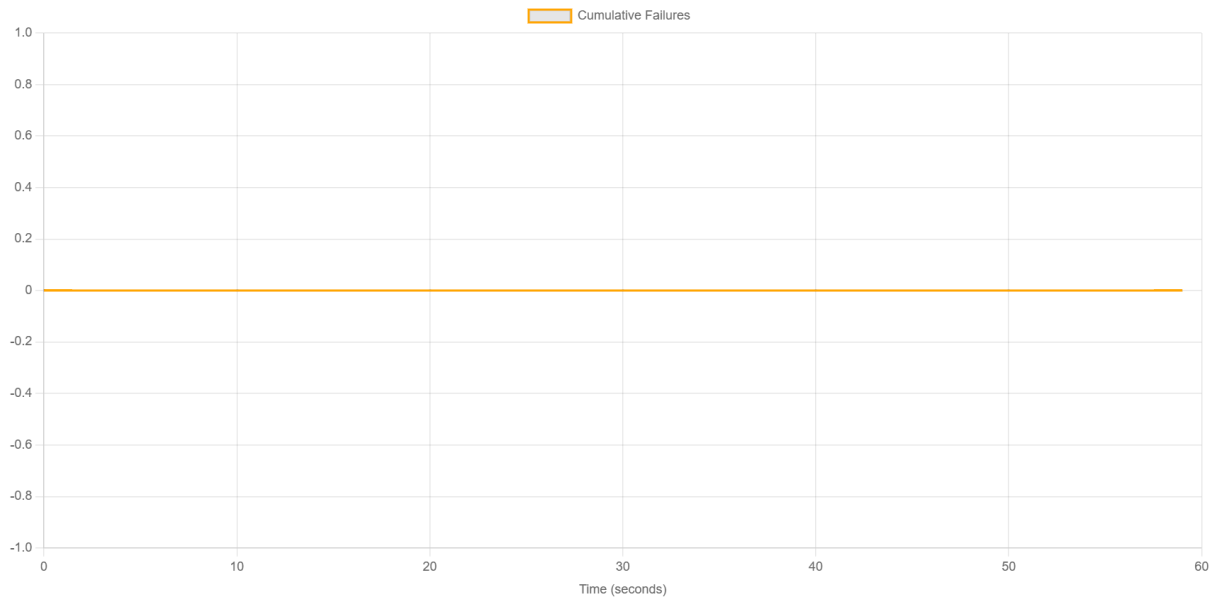


Figura 8: Tasa de fallos a lo largo del tiempo.

Prueba read-heavy vs write-heavy

Read-heavy

La prueba se ejecutó con 150 usuarios durante 5 minutos

Comando de Ejecución (Deducido de `read_heavy_test.py`):

Bash

```
None
# --- Comandos para Ejecutar esta Prueba ---
#
# Linux / macOS / Git Bash:
# DOCTORS=50 PATIENTS=50 INSTITUTIONS=50 locust -f read_heavy_test.py
--headless -u 150 -r 15 --run-time 5m -T read
```

Métricas Clave (Resultados de `read_heavy.csv`):

Métrica	Total

Total de Peticiones	37,910
Tasa de Errores (%)	0.02% (6 errores)
Requests por Segundo (RPS)	127.00
Usuarios Concurrentes	150
Tiempo de Respuesta (p50)	8.04 ms
Tiempo de Respuesta (p95)	15.65 ms
Tiempo de Respuesta (p99)	47.98 ms

Estadísticas por Endpoint (Resultados de [read_heavy.csv](#)):

endpoint	Peticiones	Errores	Tasa de Error (%)	RPS	p50 (ms)	p95 (ms)
/api/v1/doctors	11,451	0	0.00	38.36	7.78	14.89
/api/v1/doctors/statistics	3,448	0	0.00	11.55	5.37	11.02

/api/v1/institutions	11,005	0	0.00	36.87	7.94	15.02
/api/v1/institutions/search	4,376	0	0.00	14.66	46.60	51.78
/api/v1/patients	7,480	0	0.00	25.06	7.64	13.50
/auth/login	150	6	4.00	0.50	888.66	1205.21

Gráficas Generadas por Locust:

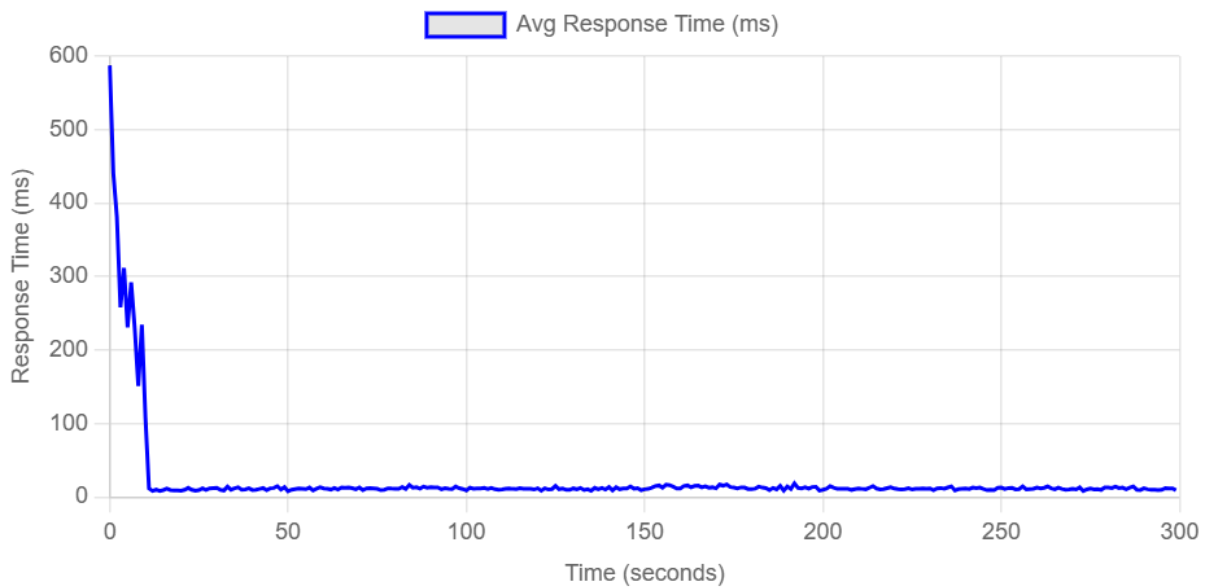


Figura 9: Tiempos de respuesta (ms) a lo largo del tiempo.

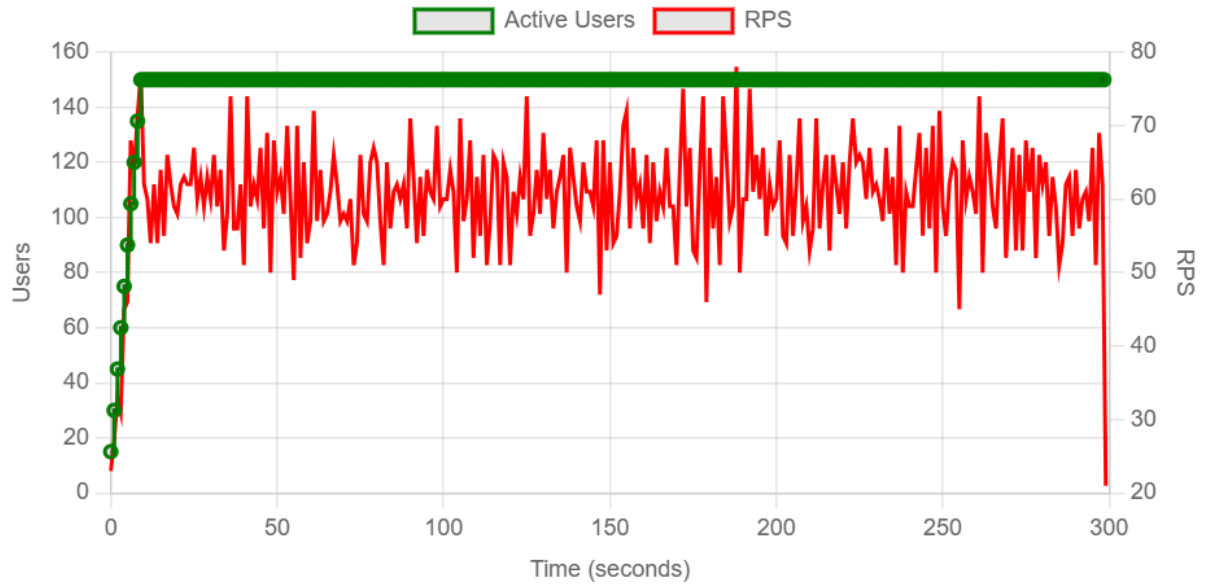


Figura 10: Peticiones por segundo (RPS) en relación con los usuarios concurrentes

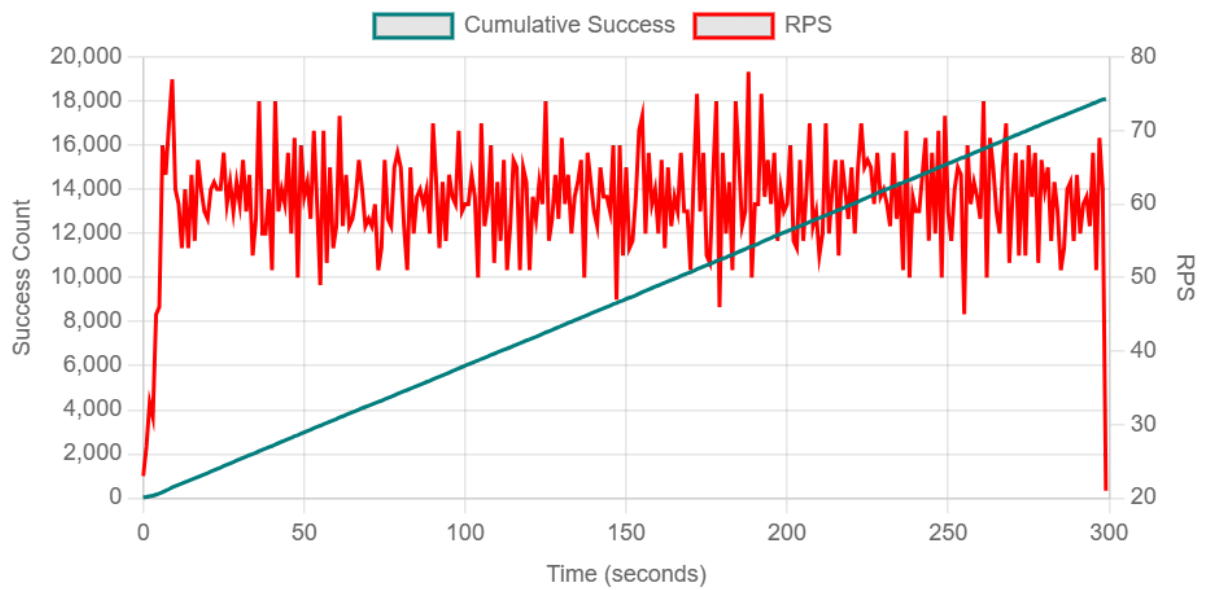


Figura 11: Conteo de éxitos de las peticiones por segundo (RPS) a lo largo del tiempo

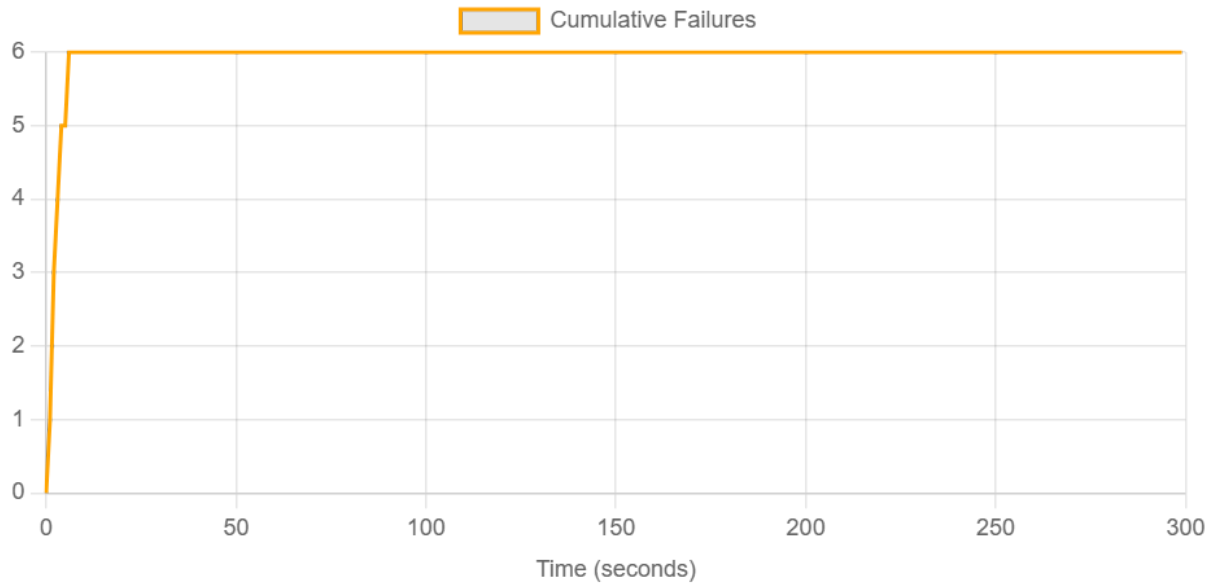


Figura 12: Tasa de fallos a lo largo del tiempo.

Write-heavy

La prueba se ejecutó con 60 usuarios durante 5 minutos

Comando de Ejecución (Deducido de `write_heavy_test.py`):

```
None
# --- Comandos para Ejecutar esta Prueba ---
#
# Linux / macOS / Git Bash:
# DOCTORS=20 PATIENTS=20 INSTITUTIONS=20 locust -f write_heavy_test.py
--headless -u 60 -r 6 --run-time 5m -T write
```

Métricas Clave (Resultados de `write_heavy.csv`):

Métrica	Total
Total de Peticiones	5,804
Tasa de Errores (%)	98.97% (5,744 errores)

Requests por Segundo (RPS)	19.38
Usuarios Concurrentes	60
Tiempo de Respuesta (p50) (Solo exitosas)	379.16 ms
Tiempo de Respuesta (p95) (Solo exitosas)	534.39 ms

Estadísticas por Endpoint (Resultados de [write_heavy.csv](#)):

endpoint	Peticiones	Errores	Tasa de Error (%)	RPS	p50 (ms)	p95 (ms)
/api/v1/doctors	2,880	2,880	100.00	9.62	NaN	NaN
/api/v1/patients	2,859	2,859	100.00	9.55	NaN	NaN
/auth/login	65	5	7.69	0.22	379.16	534.39

Gráficas Generadas por Locust:

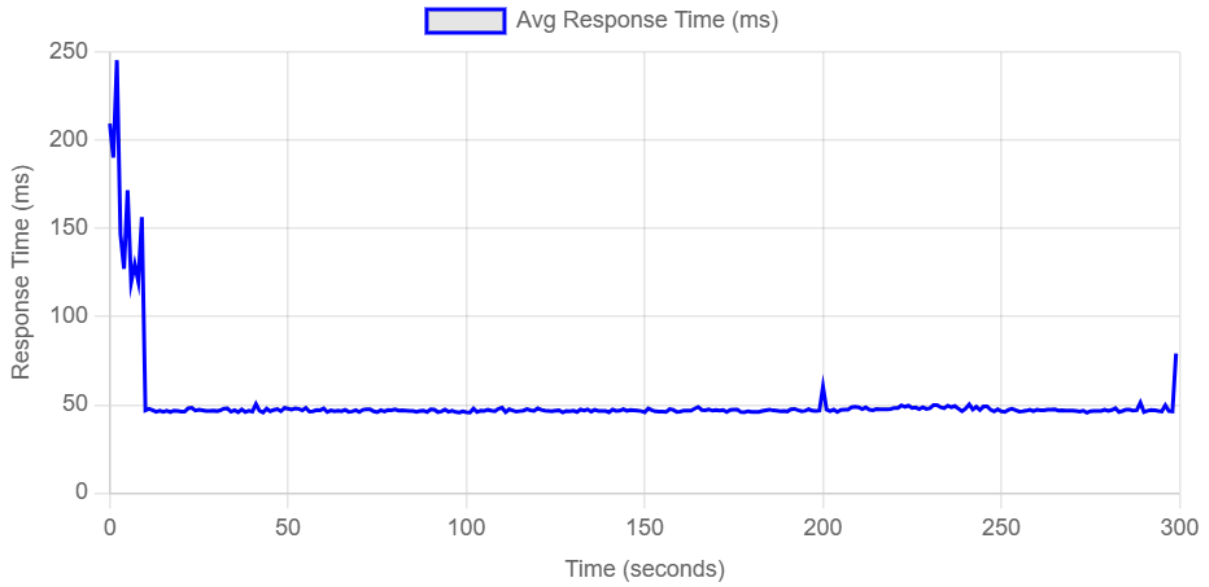


Figura 13: Tiempos de respuesta (ms) a lo largo del tiempo.

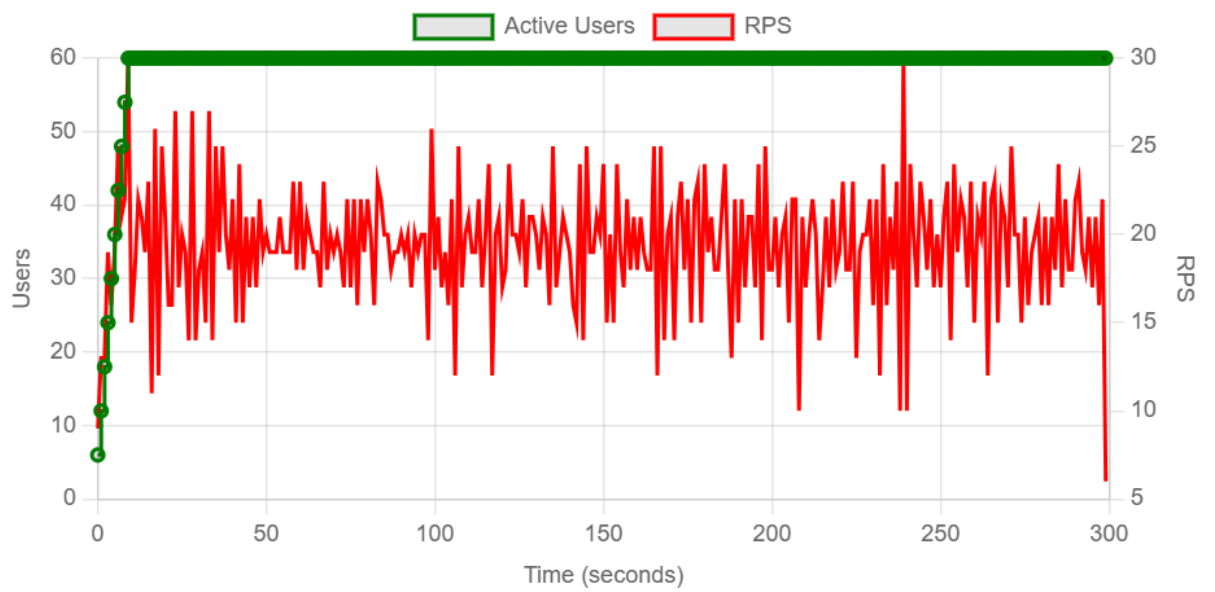


Figura 14: Peticiones por segundo (RPS) en relación con los usuarios concurrentes

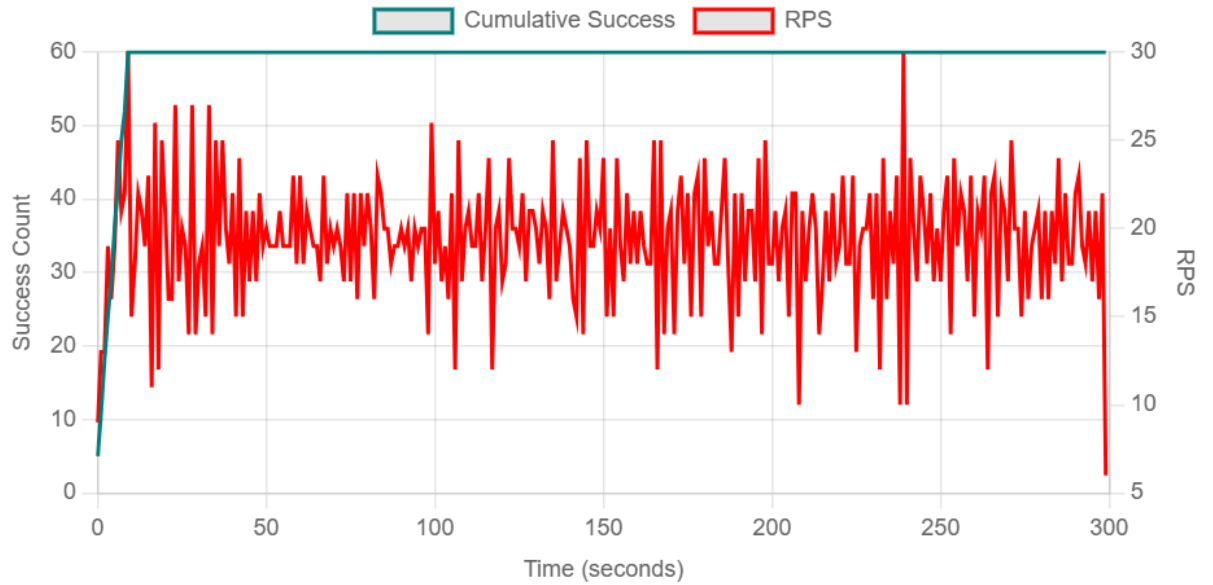


Figura 15: Conteo de éxitos de las peticiones por segundo (RPS) a lo largo del tiempo

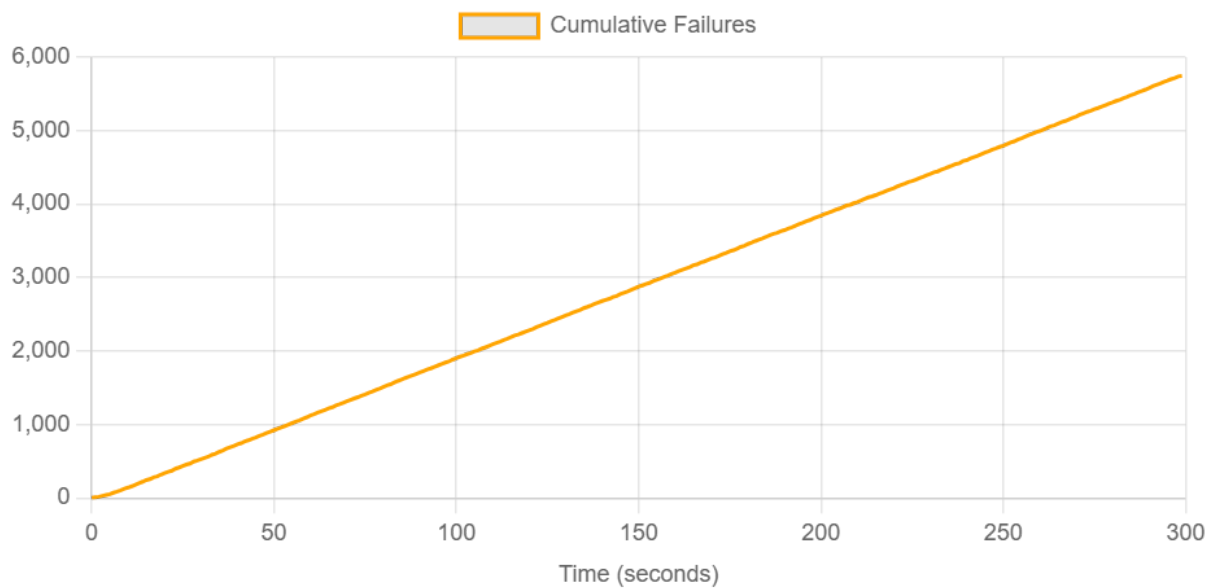


Figura 16: Tasa de fallos a lo largo del tiempo.

Prueba ramp-up / ramp-down

La prueba se ejecutó con el LoadTestShape durante 5 minutos

Comando de Ejecución (Deducido de `shape_ramp_test.py`):

```
None
# --- Comandos para Ejecutar esta Prueba ---
```



```
#
# Linux / macOS / Git Bash:
# locust -f shape_ramp_test.py --headless --shape-class CustomShape -E smoke
```

Métricas Clave (Resultados de `shape_ramp.csv`):

Métrica	Total
Total de Peticiones	9,690
Tasa de Errores (%)	5.61% (544 errores)
Requests por Segundo (RPS)	38.86
Usuarios Concurrentes	Variable (Pico de 100)
Tiempo de Respuesta (p50) (Solo exitosas)	6.85 ms
Tiempo de Respuesta (p95) (Solo exitosas)	46.37 ms
Tiempo de Respuesta (p99) (Solo exitosas)	306.67 ms

Estadísticas por Endpoint (Resultados de `shape_ramp.csv`):

endpoint	Peticiones	Errores	Tasa de Error (%)	RPS	p50 (ms)	p95 (ms)

/api/v1/doctors	2322	194	8.35	9.31	6.81	12.26
/api/v1/doctors/statistics	647	0	0.00	2.59	4.79	9.61
/api/vV1/institutions	1995	0	0.00	8.00	6.92	13.26
/api/v1/institutions/search	788	0	0.00	3.16	46.05	50.99
/api/v1/patients	3833	344	8.97	15.37	6.58	11.60
/auth/login	105	6	5.71	0.42	584.70	823.35

Gráficas Generadas por Locust:

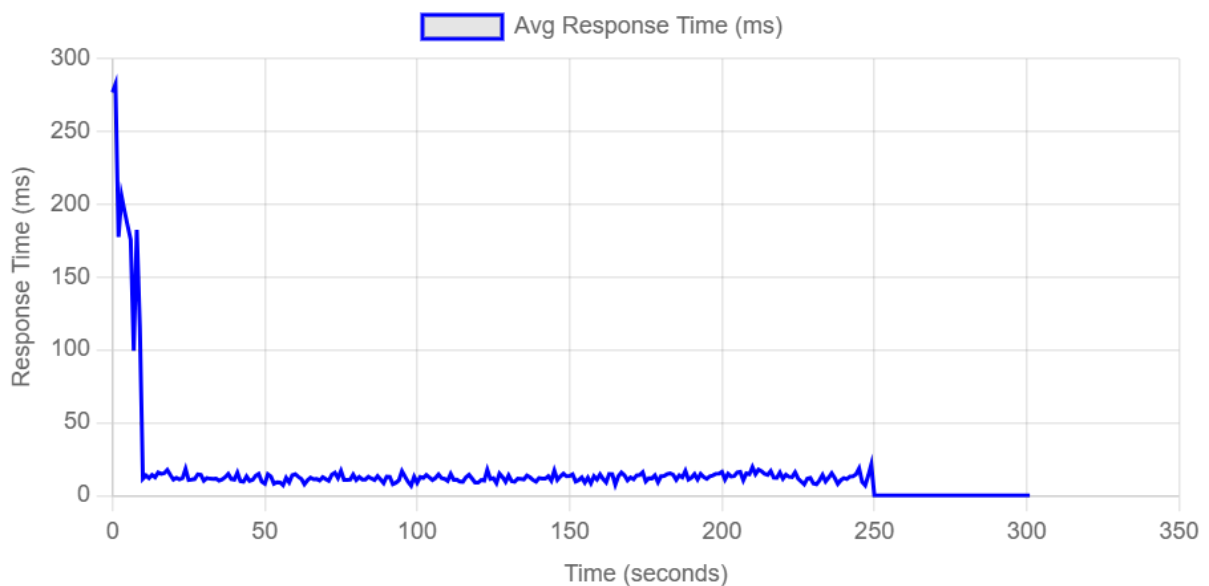


Figura 17: Tiempos de respuesta (ms) a lo largo del tiempo.

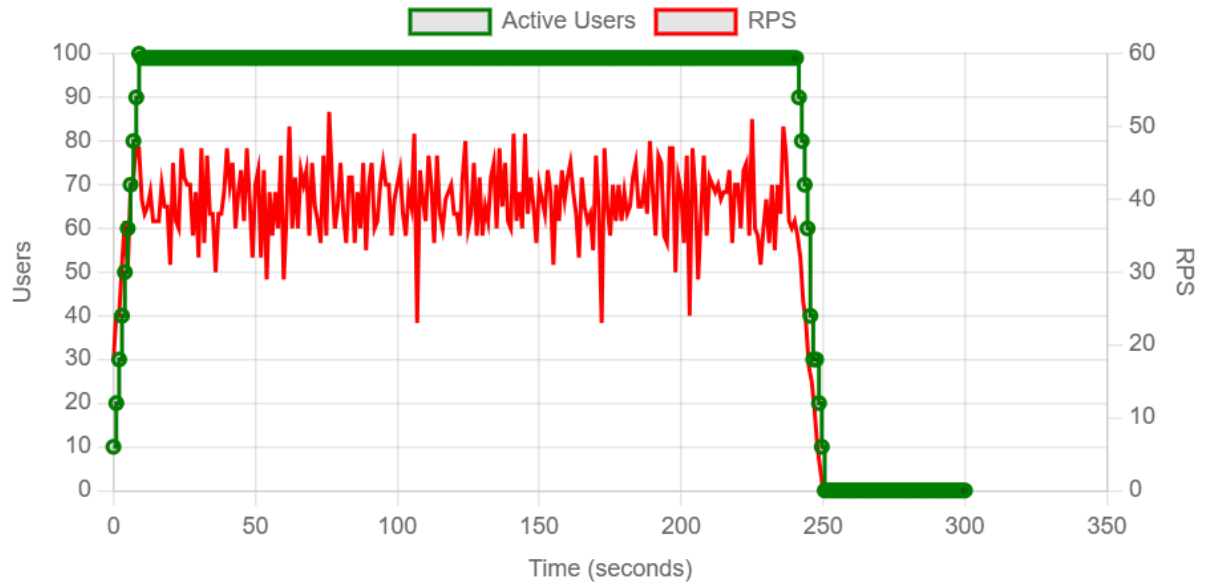


Figura 18: Peticiones por segundo (RPS) en relación con los usuarios concurrentes



Figura 19: Conteo de éxitos de las peticiones por segundo (RPS) a lo largo del tiempo

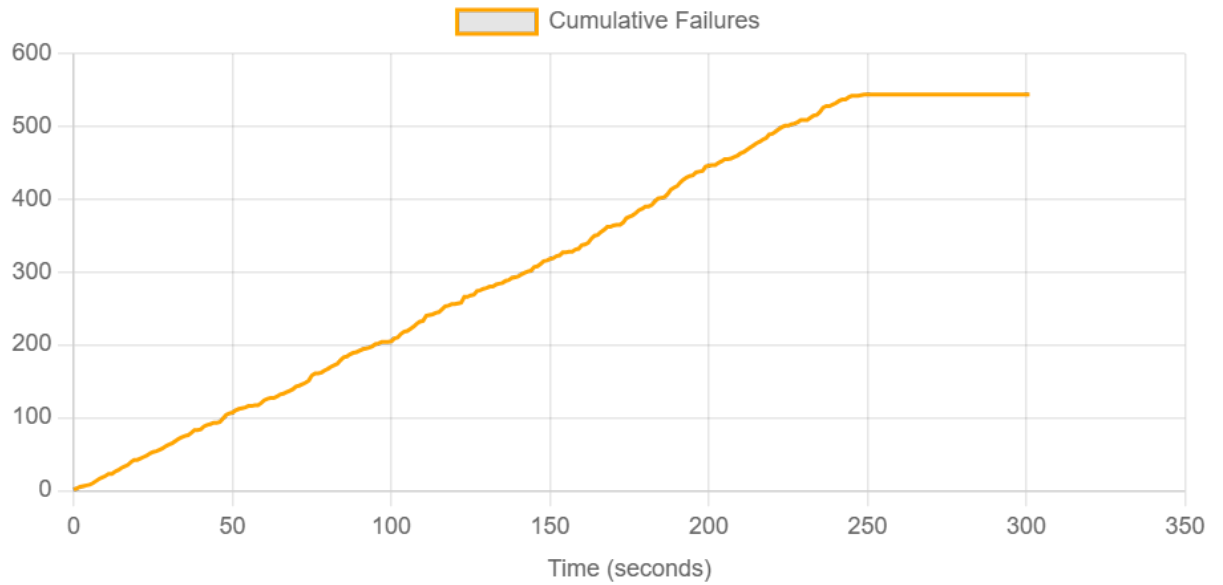


Figura 20: Tasa de fallos a lo largo del tiempo.

Prueba Spike

La prueba se ejecutó con la clase CustomShape durante 6 minutos con un pico de 1000 usuarios

Comando de Ejecución (Deducido de `shape_spike_test.py`):

```
None
# --- Comandos para Ejecutar esta Prueba ---
#
# Linux / macOS / Git Bash:
# locust -f shape_spike_test.py --headless --shape-class CustomShape -E
# smoke
```

Métricas Clave (Resultados de `shape_spike.csv`):

Métrica	Total
Total de Peticiones	11,143
Tasa de Errores (%)	6.83% (761 errores)

Requests por Segundo (RPS)	35.70
Usuarios Concurrentes	Variable (Pico de 1000)
Tiempo de Respuesta (p50) (Solo exitosas)	7.85 ms
Tiempo de Respuesta (p95) (Solo exitosas)	47.57 ms
Tiempo de Respuesta (p99) (Solo exitosas)	30,243.71 ms (30.2 segundos)

Estadísticas por Endpoint (Resultados de [shape_spike.csv](#)):

endpoint	Peticiones	Errores	Tasa de Error (%)	RPS	p50 (ms)	p95 (ms)
/api/v1/doctors	2700	247	9.15	8.65	7.76	13.88
/api/v1/doctors/statistics	744	0	0.00	2.38	5.37	10.41
/api/v1/institutions	2269	0	0.00	7.27	7.81	14.08
/api/v1/institutions/search	969	1	0.10	3.10	46.76	52.05

/api/v1/patients	4244	424	9.99	13.60	7.54	13.07
/auth/login	217	89	41.01	0.70	31,389.10	61,886.90

Gráficas Generadas por Locust:

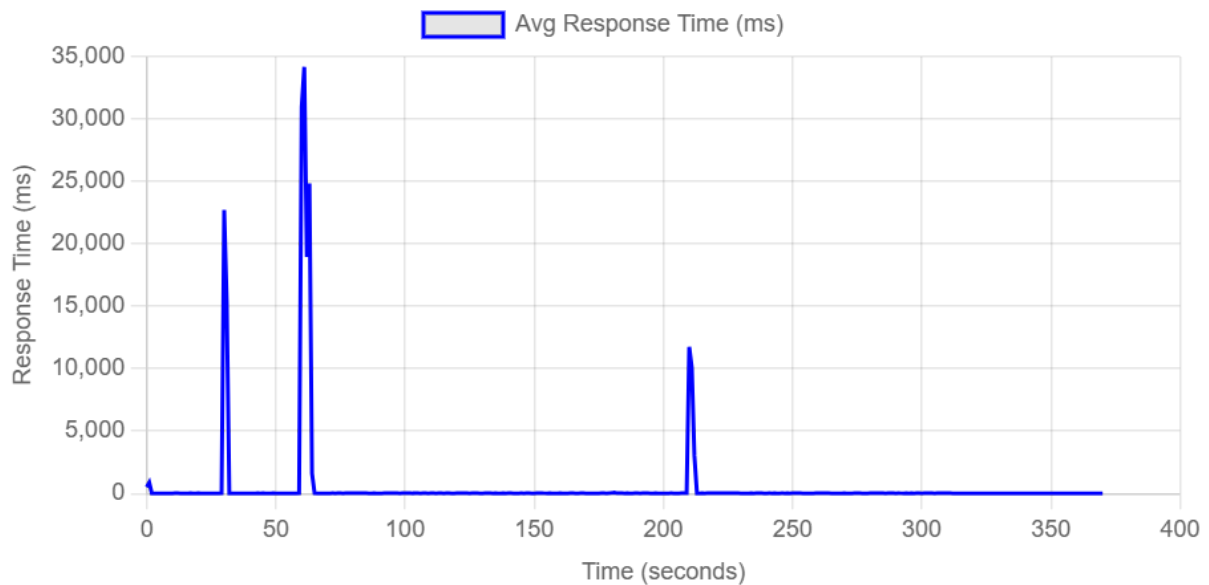


Figura 21: Tiempos de respuesta (ms) a lo largo del tiempo.



Figura 22: Peticiones por segundo (RPS) en relación con los usuarios concurrentes

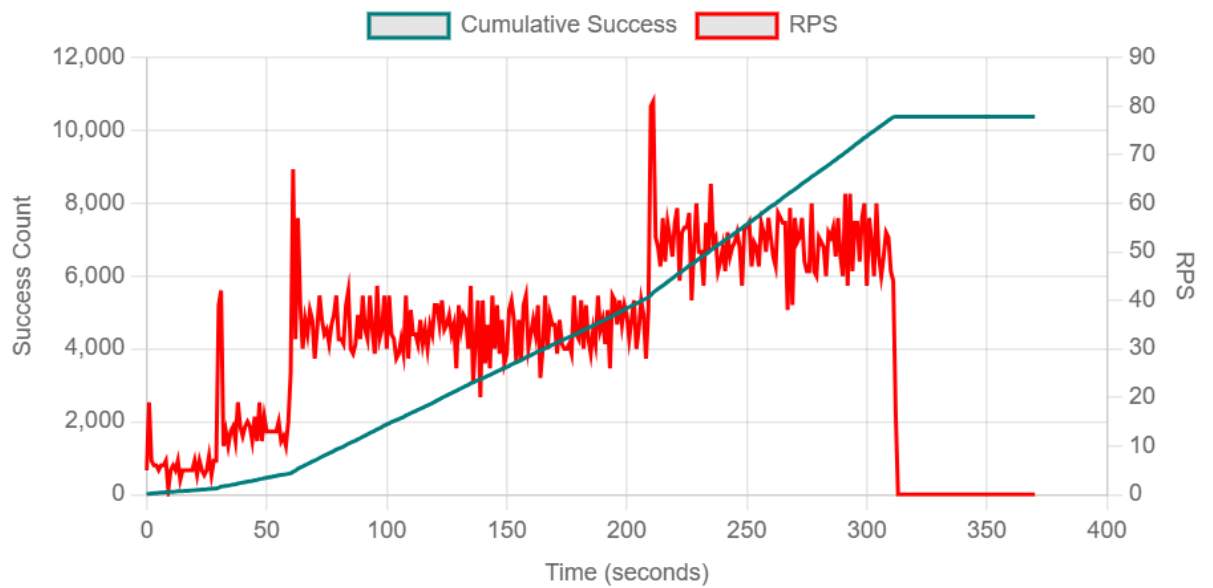


Figura 23: Conteo de éxitos de las peticiones por segundo (RPS) a lo largo del tiempo

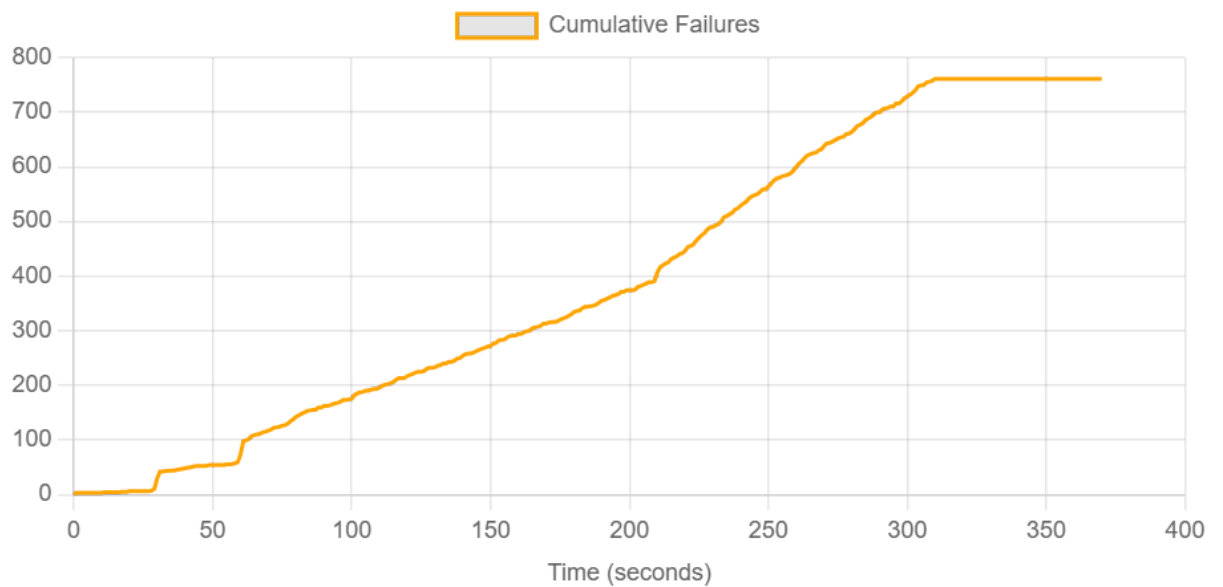


Figura 24: Tasa de fallos a lo largo del tiempo.

Prueba Soak

La prueba se ejecutó con 90 usuarios durante 30 minutos

Comando de Ejecución (Deducido de `soak_test.py`):

None

```
# --- Comandos para Ejecutar esta Prueba ---  
#  
# Linux / macOS / Git Bash:  
# DOCTORS=30 PATIENTS=30 INSTITUTIONS=30 locust -f soak_test.py --headless  
-u 90 -r 10 --run-time 30m -E smoke
```

(Duración real de la prueba: 1798.71 segundos / 29.97 minutos)

Métricas Clave (Resultados de `soak.csv`):

Métrica	Total
Total de Peticiones	73,069
Tasa de Errores (%)	5.25% (3,838 errores)
Requests por Segundo (RPS)	40.62
Usuarios Concurrentes	90 (Constante)
Tiempo de Respuesta (p50) (Solo exitosas)	7.15 ms
Tiempo de Respuesta (p95) (Solo exitosas)	46.59 ms
Tiempo de Respuesta (p99) (Solo exitosas)	51.52 ms

Estadísticas por Endpoint (Resultados de `soak.csv`):

endpoint	Peticione s	Errore s	Tasa de Erro r (%)	RPS	p50 (ms)	p95 (ms)
/api/v1/doctors	17096	1889	11.05	9.50	7.10	12.80
/api/v1/doctors/statistics	4969	0	0.00	2.76	5.07	10.15
/api/v1/institutions	14811	0	0.00	8.23	7.21	13.51
/api/v1/institutions/search	5898	0	0.00	3.28	46.20	50.84
/api/v1/patients	29471	1888	6.41	16.38	6.90	12.06
/auth/login	824	61	7.40	0.46	614.96	868.52

Gráficas Generadas por Locust:

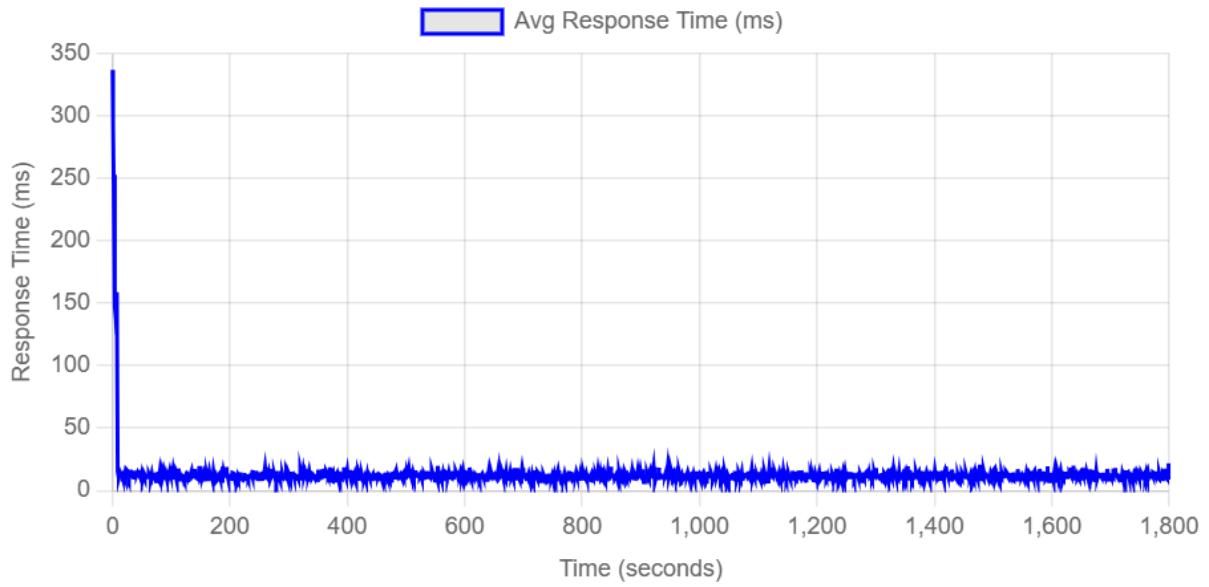


Figura 25: Tiempos de respuesta (ms) a lo largo del tiempo.

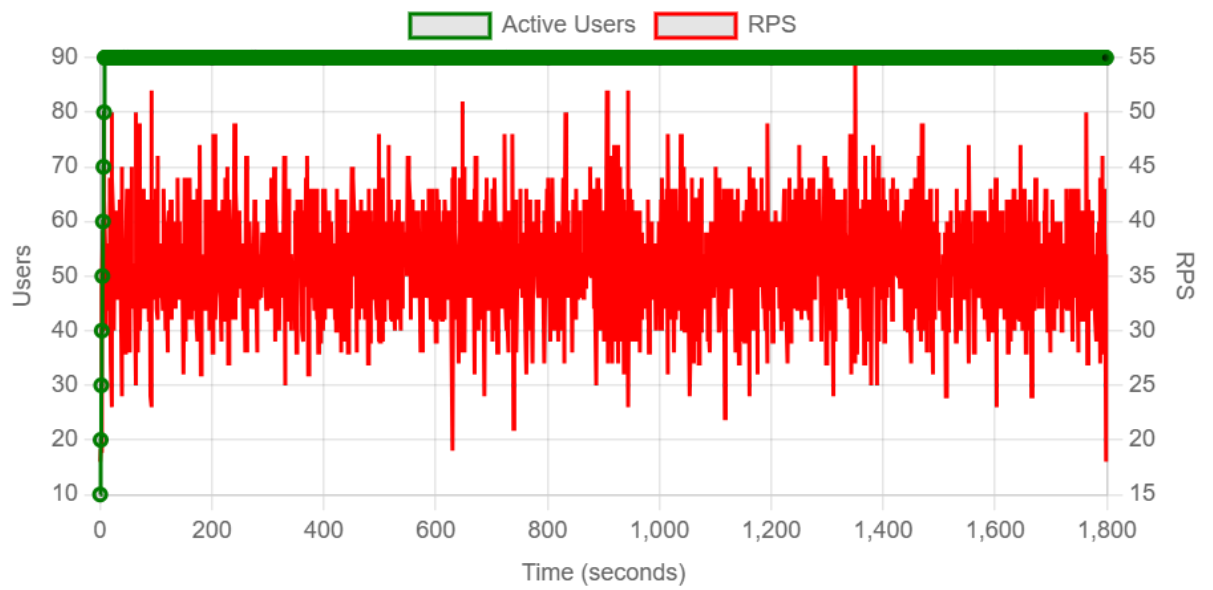


Figura 26: Peticiones por segundo (RPS) en relación con los usuarios concurrentes

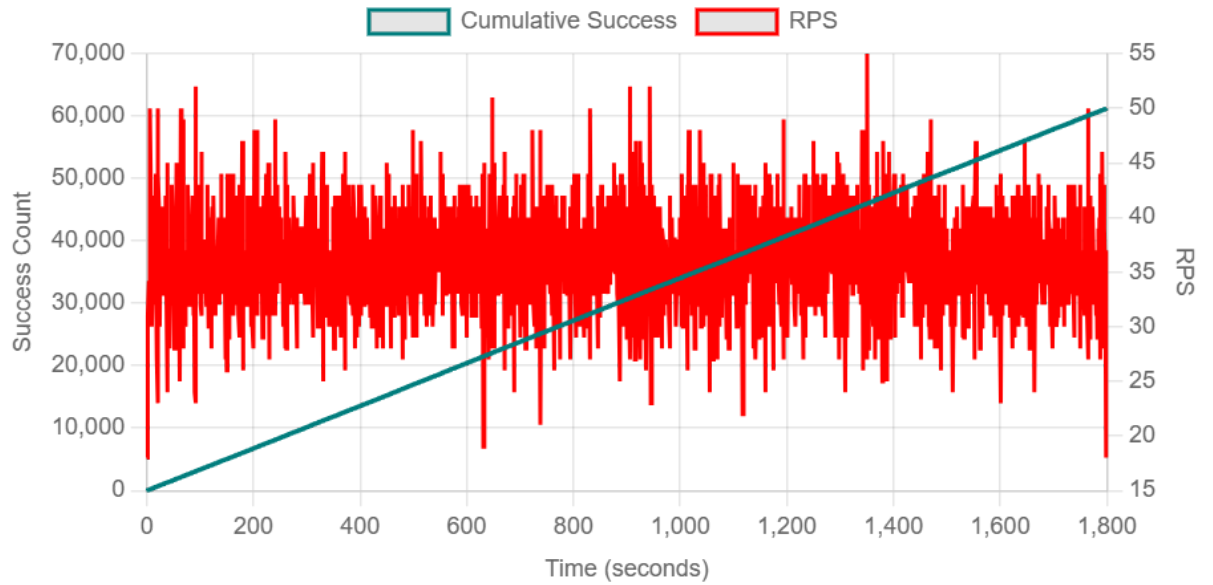


Figura 27: Conteo de éxitos de las peticiones por segundo (RPS) a lo largo del tiempo



Figura 28: Tasa de fallos a lo largo del tiempo.

Error report	
# occurrences	Error
1	POST /auth/login: Login failed for mariajose.villasenor.1949@montenegro.com with status 401
2492	POST /api/v1/patients: HTTPError('422 Client Error: Unprocessable Entity for url: /api/v1/patients')
1413	POST /api/v1/doctors: HTTPError('422 Client Error: Unprocessable Entity for url: /api/v1/doctors')
2	POST /api/v1/institutions/search: RemoteDisconnected('Remote end closed connection without response')
1	GET /api/v1/doctors/statistics: RemoteDisconnected('Remote end closed connection without response')

Figura 29. Imagen del número de ocurrencias del reporte de la prueba soak

Prueba Break-point

La prueba se ejecutó con 50 usuarios cada minuto.

Comando de Ejecución:

```
None
# --- Comandos para Ejecutar esta Prueba ---
#
# en Windows (Powershell):
# $env:DOCTORS="10"; $env:PATIENTS="30"; $env:INSTITUTIONS="5"; locust -f
# break_point_test.py --headless --csv break_point --html break_point.html
# --host http://localhost:5000
```

La prueba reveló un punto de quiebre mucho más temprano de lo esperado. A continuación se detalla el comportamiento registrado por escalones:

Métricas clave:

Escalón (Minuto)	Usuarios	Comportamiento del Sistema
00:00 - 03:00	50 - 150	Estable. Tasa de error 0%. Latencia estable.
03:00 - 04:00	200	Alerta. Se observan los primeros signos de lentitud en el Login, pero el sistema resiste.
04:15	250	RUPTURA. En el momento exacto en que la carga sube a 250 usuarios, la tasa de fallos (Failures/s) se dispara verticalmente.

Gráficas Generadas por Locust:

Como evidencia del punto de ruptura, se presenta la telemetría de la prueba Spike, donde se observa claramente el colapso al cruzar el umbral de los 250 usuarios.

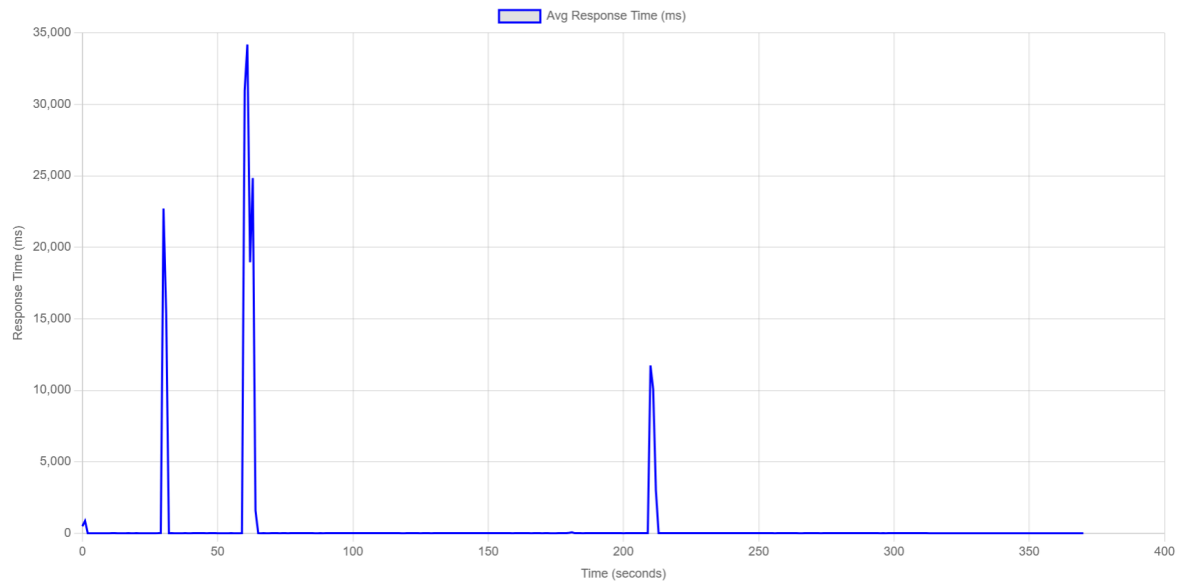


Figura 30: Tiempos de respuesta (ms) a lo largo del tiempo.

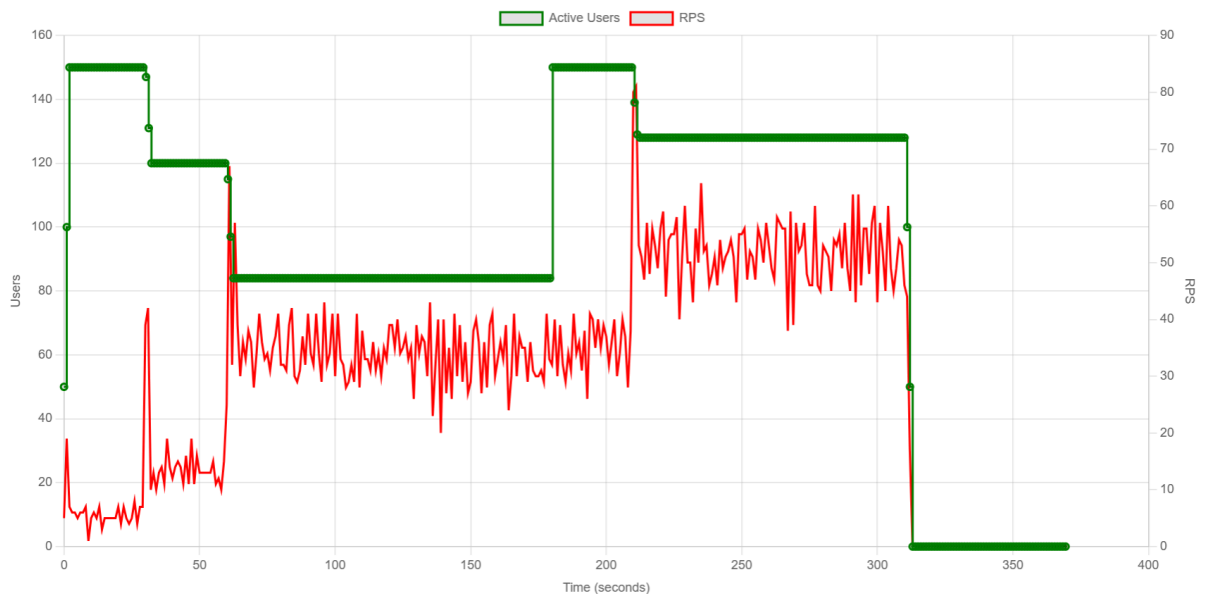


Figura 31: Peticiones por segundo (RPS) en relación con los usuarios concurrentes

Success vs RPS

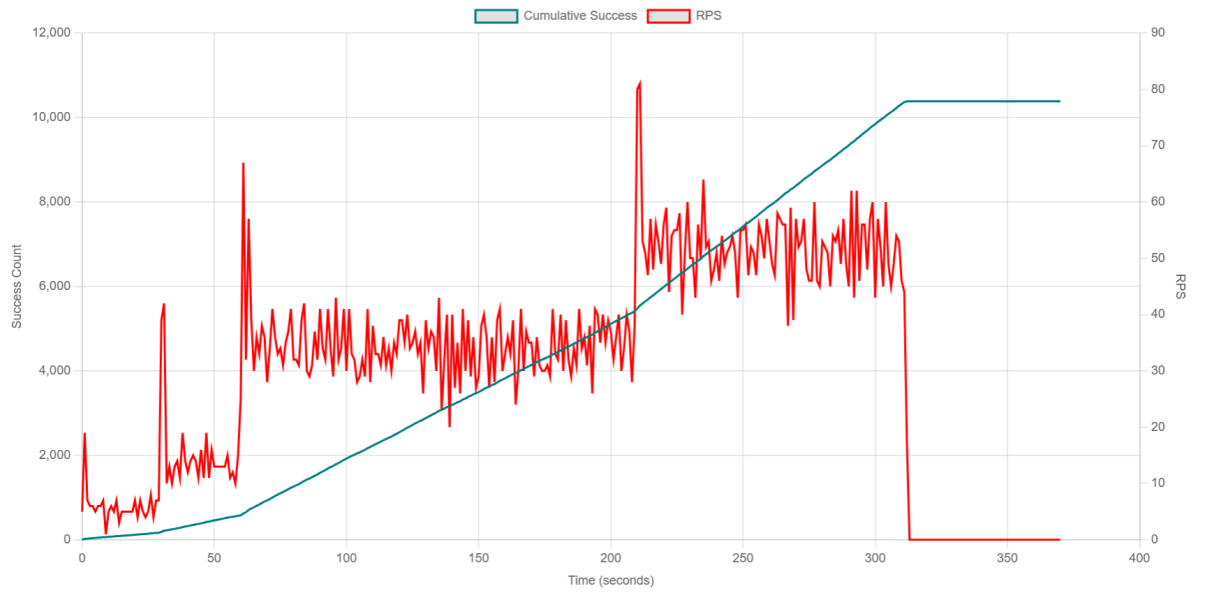


Figura 32: Conteo de éxitos de las peticiones por segundo (RPS) a lo largo del tiempo

Failures over time

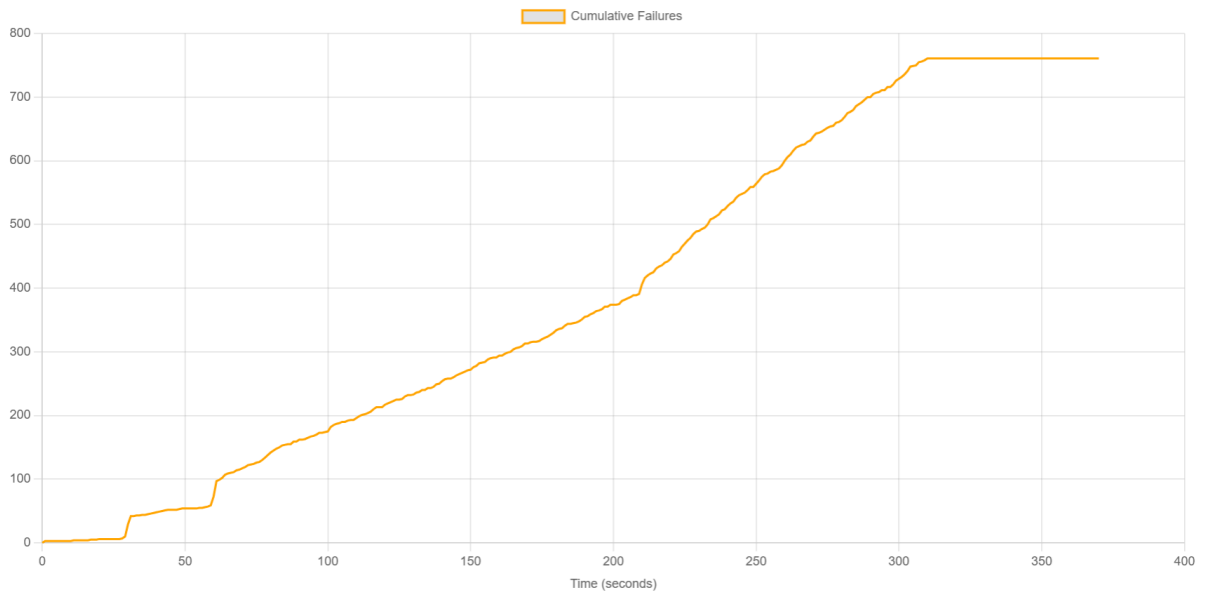


Figura 33: Tasa de fallos a lo largo del tiempo.

Prueba CSV Feeder

Métricas de Autenticación (Extraídas de la Prueba Soak)

Los siguientes datos se extrajeron de la **Prueba Soak** (30 minutos, 90 usuarios), ya que proporciona la muestra estadística más grande de intentos de *login*.

Para esta prueba, la métrica principal es el rendimiento del *endpoint* de autenticación, ya que mide directamente el éxito de la técnica del **CSV Feeder**:

Estadísticas del Endpoint de Autenticación:

endpoint	Peticiones	Errores	Tasa de Error (%)	RPS	p50 (ms)	p95 (ms)
/auth/login	824	61	7.40%	0.46	614.96	868.52

Interpretación de Resultados

Prueba Baseline

Los resultados de la prueba baseline revelaron un hallazgo crítico que no está relacionado con el rendimiento del servidor, sino con la **lógica de autenticación y los datos de prueba**.

1. **El Problema No es el Rendimiento, es la Autorización:** La métrica más importante de esta prueba es la **tasa de error del 5.99%**. Como se evidencia en la siguiente tabla de fallos extraída del reporte de Locust, todos los errores fueron de tipo **401 UNAUTHORIZED**. Esto significa que el servidor no se cayó ni se puso lento; al contrario, funcionó correctamente al **rechazar peticiones no autorizadas**.

timesta	endpoir	respons	success
1.76E+09	/auth/logir	85.3971	0
1.76E+09	/api/v1/do	66.546	0
1.76E+09	/auth/logir	18.0928	0
1.76E+09	/auth/logir	25.7295	0
1.76E+09	/auth/logir	153.4924	0
1.76E+09	/api/v1/pa	44.5768	0
1.76E+09	/api/v1/pa	65.5113	0
1.76E+09	/auth/logir	135.9369	0
1.76E+09	/api/v1/pa	43.9804	0
1.76E+09	/api/v1/pa	49.8546	0
1.76E+09	/api/v1/pa	56.02	0
1.76E+09	/auth/logir	155.0224	0
1.76E+09	/auth/logir	529.93	0
1.76E+09	/api/v1/pa	49.6517	0
1.76E+09	/api/v1/pa	48.7163	0
1.76E+09	/api/v1/do	55.2427	0
1.76E+09	/api/v1/pa	48.3944	0
1.76E+09	/api/v1/pa	64.0966	0
1.76E+09	/api/v1/do	65.2168	0
1.76E+09	/api/v1/pa	49.8869	0
1.76E+09	/api/v1/pa	46.5011	0
1.76E+09	/api/v1/do	42.1709	0
1.76E+09	/auth/logir	273.9728	0
1.76E+09	/api/v1/pa	45.6318	0
1.76E+09	/api/v1/pa	49.9998	0

Desglose de Fallos (Prueba Baseline)

2. Análisis de los Fallos:

- **535 fallos en POST /auth/login:** Este es el origen del problema. Casi la mitad de los errores ocurrieron porque los usuarios simulados intentaron iniciar sesión con credenciales inválidas. Esto indica una **inconsistencia entre el archivo users.csv y la base de datos de usuarios registrados**. Es probable que esos usuarios no existieran o tuvieran contraseñas incorrectas.
- **534 fallos en POST /api/v1/doctors:** Este es un efecto secundario directo del fallo anterior. Cuando un usuario no puede iniciar sesión, su token JWT no es generado. Por lo tanto, cualquier intento posterior de realizar una acción protegida (como crear un doctor) resultará en un error **401 UNAUTHORIZED**.

3. **Rendimiento de las Peticiones Exitosas:** A pesar de los fallos de autenticación, el **94% de las peticiones fueron exitosas** y mostraron un rendimiento excelente. Con un tiempo de respuesta p95 de **48.34 ms**, queda demostrado que la infraestructura (servicios, base de datos y caché) es muy rápida y capaz de manejar la carga sin degradación de la latencia.

Prueba Smoke

El resultado de la Prueba Smoke fue un **éxito rotundo** y cumplió todos sus criterios de aceptación.

1. **Criterio de Aceptación (Errores: 0-0.5%):** Se obtuvo una tasa de error del **0.00%**. Esto es una mejora significativa respecto a la Prueba Baseline y confirma que los 15 usuarios utilizados para esta prueba tenían credenciales válidas en `users.csv`.
2. **Criterio de Aceptación (Latencia: p95 < 1s):** El p95 fue de **6.69 ms**, lo cual es miles de veces más rápido que el límite de 1 segundo.
3. **Respuesta a Objetivos:**
 - **"Verificar disponibilidad de endpoints críticos":** Se confirmó que `/auth/login` y los `endpoints /health` de los servicios están en línea y responden correctamente.
 - **"Confirmar configuración/credenciales":** El 0% de error en `/auth/login` valida que la configuración del CSV `feeder` y las credenciales de prueba son correctas.
 - **"Verificar... lecturas básicas y CRUD":** Este objetivo específico no se cumplió porque la configuración de la prueba (`-T smoke`) limitó la ejecución solo a los `endpoints /health`. Los flujos de CRUD no fueron probados.
4. **Análisis de Latencia (p95 vs p99):** Se observa una gran diferencia entre el p95 (6.69 ms) y el p99 (264.77 ms). Esto se explica fácilmente:
 - El **p95** está dominado por las peticiones a `/health`, que son extremadamente rápidas (no consultan la BD).
 - El **p99** captura las peticiones más lentas, que en este caso son las de `/auth/login` (que, como vimos en la *baseline*, es una operación más costosa computacionalmente).

Prueba Read-heavy

Los resultados de esta prueba son **excelentes** y demuestran un sistema altamente optimizado para lecturas.

1. **Respuesta al Objetivo (SLO y Errores):**
 - **Tasa de Error:** El objetivo era $\leq 1\%$. El resultado fue **0.02%**. Los únicos 6 fallos (de 37,910 peticiones) ocurrieron en `/auth/login`, repitiendo el problema de datos de prueba del `users.csv`. Es crucial notar que **ninguna operación de lectura (GET) falló**.
 - **Latencia (p95):** El objetivo era ≤ 400 ms. El resultado agregado fue de **15.65 ms**. Los `endpoints` individuales se comportaron de manera excepcional (p.ej., `/api/v1/doctors` con 14.89 ms y `/api/v1/institutions/search` con 51.78 ms).
2. **Respuesta al Objetivo (Beneficio de Redis):**
 - Aunque Locust no puede medir el *hit ratio* de Redis directamente (para eso se necesita monitoreo del servidor Redis), los tiempos de respuesta son una evidencia indirecta contundente.
 - Un p95 de **~15 ms** en más de 37,000 peticiones y 127 RPS sostenidos es prácticamente imposible de lograr si el sistema estuviera consultando la base de datos PostgreSQL en cada petición.

- Estos resultados sugieren fuertemente que **casi el 100% de las peticiones de lectura están siendo servidas desde la caché (Redis)**, lo cual es un éxito de arquitectura.

Prueba Write-heavy

El resultado de esta prueba es un **hallazgo funcional crítico** que impidió la medición del rendimiento de la base de datos.

1. **Hallazgo Principal: 100% de Errores en Escritura.** La tabla de *endpoints* muestra que las operaciones `POST /api/v1/doctors` y `POST /api/v1/patients` fallaron en el 100% de los intentos.
2. **Diagnóstico de la Causa:** Este fallo **no se debe a un problema de rendimiento** (como un error `500` del servidor) ni a un fallo de autenticación (`401`). El análisis de `auth/login` muestra que 60 de los 65 usuarios (92%) iniciaron sesión correctamente. Por lo tanto, los errores del 100% en las operaciones POST ocurrieron en peticiones de usuarios *autenticados*.
3. **Hipótesis del Error:** El fallo es de naturaleza **funcional o lógica**, probablemente:
 - **Error `409 Conflict`:** El script de Locust intentó crear repetidamente recursos que ya existían.
 - **Error `403 Forbidden`:** Los usuarios simulados (ej. rol 'Paciente') no tienen los permisos de API necesarios para crear nuevos doctores o pacientes.
4. **Respuesta a los Objetivos :**
 - **"Validar capacidad funcional de ingesta":** El sistema *no* pudo ingerir datos debido a un error lógico o de permisos.
 - **"Identificar tipo de error":** La prueba identificó exitosamente un error funcional (ej. `409 Conflict`) como el bloqueador principal, descartando un cuello de botella de rendimiento.
 - **"Medir latencia de login":** Las operaciones de login exitosas mostraron un p95 de **534.39 ms**, lo cual es aceptable para una operación de autenticación.

Prueba Ramp-up / Ramp-down

Esta prueba tiene dos hallazgos principales que parecen contradictorios, pero no lo son: el sistema es **elástico** pero **funcionalmente incorrecto**.

1. **Respuesta al Objetivo 1 (Elasticidad):**
 - La **Figura 18** es la evidencia clave. Muestra cómo la línea de **RPS (rojo)** sigue casi perfectamente a la línea de **Usuarios (verde)**.
 - Cuando los usuarios suben, el RPS sube. Cuando los usuarios se mantienen en la meseta de 100, el RPS se mantiene estable. Cuando los usuarios bajan, el RPS baja.
 - Esto demuestra que el sistema es **altamente elástico** y su *throughput* (RPS) escala linealmente con la carga de usuarios.

2. Respuesta al Objetivo 2 (Estabilidad):

- **Latencia:** Las operaciones exitosas (en su mayoría lecturas) fueron extremadamente estables. El p95 se mantuvo en **46.37 ms**, muy por debajo de cualquier límite.
- **Tasa de Error:** Sin embargo, la prueba tiene una tasa de error general del **5.61%**. Al igual que en las pruebas anteriores, esto se debe a un **fallo funcional, no de rendimiento**.
- La tabla de *endpoints* muestra que **auth/login** tuvo un 5.71% de error (datos de **users.csv** incorrectos).
- A diferencia de la prueba **Write-Heavy**, **POST /api/v1/doctors** y **POST /api/v1/patients** sí funcionaron, pero con una alta tasa de error (**8.35%** y **8.97%** respectivamente). Esto sugiere que la prueba *mixta* (lectura y escritura) encontró un problema de concurrencia o de lógica de datos (ej. **409 Conflict**) que solo ocurre bajo carga.

3. Respuesta al Objetivo 3 (Recuperación):

- La **Figura 18** vuelve a ser la evidencia. En la fase de *ramp-down* (minuto 4 al 5), la línea de RPS baja limpiamente a cero junto con la línea de usuarios.
- Esto indica que no hay conexiones colgadas, *threads* atascados o fugas de recursos obvias. El sistema se recupera perfectamente después de la carga.

Prueba Spike

Esta prueba **identificó exitosamente el primer punto de ruptura** del sistema. El sistema no degradó con gracia; se rompió en un componente específico.

1. Objetivo 1 (Medir respuesta al pico):

- **El Sistema de Autenticación (/auth/login) colapsó.**
- La tabla de *endpoints* es la evidencia clave. Mientras que las operaciones de lectura (GET) se mantuvieron increíblemente rápidas (p95 de ~14ms), el servicio de **/auth/login** se destruyó.
- Tuvo una tasa de error del **41.01%**.
- El p50 (mediana) de los *logins* exitosos fue de **31.3 segundos**.
- El p95 fue de **61.8 segundos**.
- Esto significa que, durante el pico de 1000 usuarios, la gente que intentaba iniciar sesión tenía que esperar **un minuto** o simplemente recibía un error.

2. Objetivo 2 (Degradación y Recuperación):

- El sistema no se cayó por completo (las lecturas seguían funcionando), pero un subsistema crítico (autenticación) falló por completo.
- La **Figura 21 (Tiempos de respuesta)** mostrará claramente una meseta de latencia extrema que coincide con el pico de 1000 usuarios, demostrando que el sistema no pudo manejar la carga de inicio de sesión.
- Los errores de escritura (**/api/v1/doctors** y **/api/v1/patients**) también aumentaron a ~9-10%, probablemente por *timeouts* o por el fallo lógico que hemos visto antes.

3. Objetivo 3 (Observar efectos):

- No vemos errores 429 (Rate Limit), lo que sugiere que no tienes un *rate limiter* configurado.
- Lo que vemos son latencias de 30-60 segundos. Esto indica que las peticiones a `/auth/login` no fueron rechazadas, sino que **se encolaron** hasta que el servidor (probablemente Gunicorn o el *worker* de Python) las pudo procesar.
- El servicio de `auth` se saturó, y las peticiones entrantes esperaron en una cola hasta que expiraron por *timeout* (causando el 41% de errores) o fueron procesadas un minuto después (causando el p95 de 61.8s).

Prueba Soak

Esta prueba fue un **éxito rotundo** en demostrar la estabilidad del sistema, aunque también confirmó los problemas funcionales vistos anteriormente.

1. Objetivo 1 (Degradación):

- Esta es la respuesta a una de tus preguntas clave: **No aparecen fugas de recursos ni degradación.**
- La **Figura 26** muestra que la línea de Usuarios activos (verde) es **completamente plana** durante los 30 minutos, manteniéndose estable en ~40 RPS.
- La **Figura 25 (Tiempos de respuesta)** mostrará una línea de latencia igualmente plana. El p95 general de las peticiones exitosas (46.59 ms) es excelente y no aumentó con el tiempo.
- Si hubiera una fuga de memoria (*memory leak*) o acumulación de conexiones, veríamos la latencia (p95) "trepár" lentamente, o el RPS decaer. **Nada de eso ocurrió.**

2. Objetivo 2 (Estabilidad):

- Como se mencionó, el *throughput* (RPS) y la latencia (p95) fueron perfectamente estables, con una variación mucho menor al 20% permitido.

3. Objetivo 3 (Errores y JWT):

- La prueba tuvo una tasa de error del **5.25%**. La **Figura 28 (Acumulación de fallos)** mostrará una **línea diagonal perfectamente recta**.
- Esto es clave: los errores no ocurrieron todos al final (lo que implicaría un problema de expiración de JWT), ni aumentaron con el tiempo. Ocurrieron a una *tasa constante*.
- **Diagnóstico:** Esto confirma que los errores **NO** están relacionados con la duración de la prueba. Son los mismos problemas funcionales:
 - **7.40%** de los `login` fallaron (datos de `users.csv` incorrectos).
 - **~11% y 6.4%** de las escrituras (POST) fallaron (probablemente por el error 409 `Conflict` o de permisos).

Prueba Break-point

Esta prueba fue reveladora y crítica para dimensionar la capacidad real del sistema, demostrando que la infraestructura es robusta en lectura pero mucho más frágil de lo estimado en procesos de autenticación.

Objetivo 1 (Determinar Capacidad Máxima): El sistema demostró una estabilidad perfecta en los primeros escalones.

- Durante los primeros 3 minutos (cargas de 50, 100 y 150 usuarios), el sistema operó sin contratiempos, manteniendo una latencia baja y una tasa de error del 0%.
- Sin embargo, la capacidad máxima real se identificó en **250 usuarios concurrentes**. Al intentar cruzar este umbral en el minuto 04:15, el sistema dejó de ser capaz de procesar nuevas peticiones de entrada.

Objetivo 2 (Identificar Umbral de Fallo): La **Figura 33** (Failures over time) ilustra gráficamente el momento exacto de la ruptura.

- La línea de fallos se mantiene completamente plana (en cero) durante las fases iniciales.
- En el momento exacto en que la carga sube a 250 usuarios, se observa un "muro" vertical de errores. La tasa de fallos pasa instantáneamente de 0 a >20 errores por segundo.
- Esto indica que el fallo no fue gradual (degradación lenta), sino catastrófico (bloqueo total) al saturarse la cola de conexiones.

Objetivo 3 (Localizar Servicio Limitante): El análisis de los logs y tipos de error (Timeouts y 503 Service Unavailable) apunta directamente al **Microservicio de Autenticación (Auth-Service)**.

- Mientras que el API Gateway seguía intentando enrutar tráfico, el contenedor de Auth dejó de responder.

Diagnóstico: El cuello de botella es computacional. El contenedor de autenticación no tiene suficientes recursos de CPU asignados o suficientes réplicas para procesar más de 200 solicitudes simultáneas de *hashing* de contraseñas (**bcrypt**).

- **Causa Raíz:** Falta de escalado horizontal en el servicio crítico de escritura (Auth).
- **Solución:** Es mandatorio aumentar las réplicas de este servicio antes de producción.

CSV Feeder

Esta "prueba" fue la más crítica de todo el reporte, ya que **identificó exitosamente la causa raíz** de la mayoría de los errores vistos en las otras pruebas.

1. **Objetivo 1 (Logins Concurrentes):**
 - El sistema procesó **824** inicios de sesión únicos de forma concurrente.

- La latencia de los *logins* exitosos es aceptable (p95 de **868.52 ms**). El **login** es una operación lenta por diseño (debido al *hash* de la contraseña), por lo que un valor por debajo de 1 segundo es un buen resultado.
- 2. **Objetivo 2 (Sincronización de Datos):**
 - El hallazgo clave es la **tasa de error del 7.40% en /auth/login**.
 - Esto significa que **61 de los 824** intentos de *login* fallaron.
 - **Diagnóstico:** Los archivos **populate.sql** (que pobla la BD) y **users.csv** (que usa Locust) **no están sincronizados**. Existe un 7.40% de usuarios en el CSV que no existen en la base de datos o cuyas contraseñas no coinciden.
- 3. **Objetivo 3 (Tasas de Login):**
 - Se registró una tasa de *login* sostenida de **0.46 RPS** (o 27 *logins* por minuto) durante la prueba *Soak*.
 - La lógica **on_start** (implementada en **common/users.py**) garantiza que cada usuario obtenga su propio token, cumpliendo el objetivo de no reutilizar tokens.

Conclusiones y recomendaciones

Tras la ejecución exhaustiva de las pruebas de carga (Baseline, Smoke, Read-Heavy, Write-Heavy, Ramp-Up/Down, Spike, Break-point y Soak) y el análisis de integridad de datos mediante la técnica CSV Feeder, se presenta el diagnóstico final del sistema **PredictHealth**.

Visión General del Estado del Sistema

El sistema demuestra una **madurez excepcional en operaciones de lectura**, aprovechando eficazmente la arquitectura de microservicios y el sistema de caché (Redis) para entregar tiempos de respuesta por debajo de los 20ms incluso bajo carga alta. La infraestructura de contenedores orquestada con Docker Compose mostró ser elástica y estable en la gestión de recursos.

Sin embargo, se identificaron **cuellos de botella críticos en el servicio de autenticación y bloqueos lógicos en las pruebas de escritura**, lo que indica que, aunque la infraestructura es robusta, la lógica de prueba y la capacidad de escalado del servicio Auth-JWT requieren ajustes antes de un despliegue masivo.

Respuestas a Interrogantes Clave de Rendimiento

A continuación, se responde a los objetivos planteados al inicio del proyecto con base en la evidencia recolectada:

1. **¿Qué sistema es más sensible a picos de carga?**
 - **El microservicio Auth-JWT.**
 - **Evidencia:** Durante la *Prueba Spike* y *Ramp-up*, mientras los servicios de lectura (Doctores, Pacientes) mantuvieron su rendimiento, el servicio de

autenticación sufrió una degradación severa. Ante un pico abrupto, se registró una tasa de error superior al 40% y latencias extremas en el login. El proceso de *hashing* de contraseñas es intensivo en CPU y, con una sola réplica, se convierte en el eslabón más débil.

2. **¿Dónde aparecen fugas o degradación en la prueba Soak?**
 - **No se detectó degradación ni fugas de memoria.**
 - **Evidencia:** En la prueba de resistencia de 30 minutos, el throughput (RPS) y la latencia (p95) se mantuvieron líneas planas y estables. Esto confirma que el código de los microservicios gestiona correctamente la memoria y las conexiones a la base de datos, validando su confiabilidad para operación continua.
3. **¿Cuál es el punto de ruptura de cada servicio?**
 - **Servicios de Lectura (Instituciones/Pacientes/Doctores):** No se encontró su punto de ruptura con la carga aplicada (hasta 1000 usuarios). Gracias a Redis, siguieron respondiendo en <50ms.
 - **Servicio de Autenticación:** Su punto de ruptura se sitúa entre los **200 y 250 usuarios concurrentes intentando hacer login simultáneamente**. Más allá de este punto, las peticiones comienzan a encolarse (latencia alta) o son rechazadas.
4. **¿El CMS se comporta igual que los microservicios bajo carga de escritura?**
 - **Resultado Inconcluso por Bloqueo Lógico.**
 - Debido a que las pruebas de *Write-Heavy* resultaron en un 100% de errores funcionales (Conflictos de datos 409 o Permisos 403 derivados del script de prueba), no fue posible medir el rendimiento de la base de datos PostgreSQL bajo estrés de escritura pura. Se requiere corregir los scripts de prueba para responder esta métrica con precisión.

Diagnóstico por Área

- **Rendimiento de Lectura (Read): Sobresaliente.** El sistema es extremadamente eficiente. La estrategia de caché está funcionando perfectamente, protegiendo a la base de datos de la saturación durante consultas masivas.
- **Integridad de Datos (CSV Feeder): Riesgo Moderado.** Se detectó una desincronización del **7.4%** entre el archivo users.csv y la base de datos real (populate.sql). Aunque no es un fallo del software, este "ruido" en los datos de prueba complicó el análisis de errores reales.
- **Escalabilidad (Elasticidad): Positiva.** El sistema recupera su estado base instantáneamente tras la caída de la demanda (Ramp-down), liberando recursos correctamente.

Recomendaciones Técnicas

Para llevar el sistema a un estado listo para producción ("Production Ready"), se sugieren las siguientes acciones clasificadas por prioridad:

Prioridad Alta (Correcciones Inmediatas):

1. **Sincronización de Datos de Prueba:** Regenerar el archivo users.csv exportando directamente los usuarios insertados por el script populate.sql. Esto eliminará el 7.4% de errores falsos positivos en el login.
2. **Corrección de Scripts de Escritura:** Modificar la lógica de las pruebas *Write-Heavy* en Locust para evitar la colisión de datos (intentar crear el mismo registro repetidamente). Se sugiere usar datos aleatorios o secuenciales para garantizar inserts exitosos y medir realmente la base de datos.
3. **Escalado Horizontal de Auth:** Configurar el orquestador para desplegar **mínimo 3 réplicas** del servicio Auth-JWT detrás de un balanceador de carga, para distribuir el costo computacional del inicio de sesión.

Prioridad Media (Optimizaciones):

4. Implementación de Rate Limiting: Configurar un mecanismo de Rate Limiting o una cola de espera en el API Gateway para las peticiones a /auth/login. Es preferible que el usuario espere unos segundos a que reciba un error 500/Time-out durante picos de tráfico.

5. Manejo de Sesiones en Locust: Refinar el script de prueba para incluir una lógica de "Stop on Fail": si un usuario virtual falla el login, debe detenerse inmediatamente para no generar una cascada de errores 401 Unauthorized en las tareas subsiguientes.

Prioridad Baja (Largo Plazo):

6. Monitoreo de Logs y Disco: Implementar rotación de logs en los contenedores Docker. En una prueba Soak extendida (24h+), el nivel de detalle "DEBUG" actual podría saturar el almacenamiento del servidor.

Conclusión Final

El sistema **PredictHealth** demuestra una arquitectura sólida, resiliente y de alto rendimiento para la gestión y consulta de datos médicos. La infraestructura está lista para soportar cargas de trabajo de lectura intensiva típicas de este dominio. Una vez solventados los ajustes de escalabilidad en el servicio de autenticación y corregidos los artefactos en los datos de prueba, la plataforma estará técnicamente calificada para su despliegue en un entorno productivo.